CHITKARA UNIVERSITY

CSS3 and Responsive Web Design

CSS- Cascading Style Sheets

CHITKARA UNIVERSITY

- A **style sheet** is a document that **contains style information** about **one or more documents** written in markup language and used to control the styles such as **fonts, colors, size, spacing, margins etc.**

- **CSS** stands for **Cascading Style Sheets is a** style sheet language that describes the style of an HTML document.



HTML

the structural layer



CSS

the presentational layer

- Describes the **appearance, layout, and presentation** of information on a web page
- Describes ***how* information is to be displayed**, not *what* is being displayed

CSS- Cascading Style Sheets

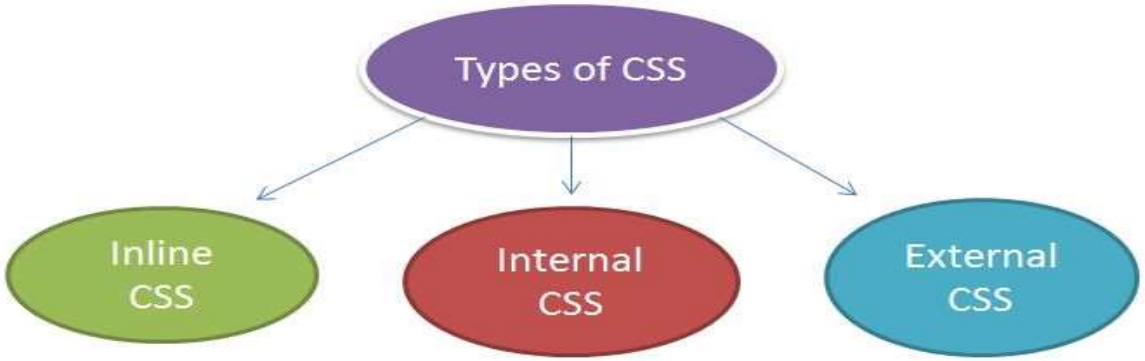
CHITKARA UNIVERSITY

- **CSS adds the styles to the HTML document**



3 WAYS TO ADD CSS TO HTML:

THREE TYPES OF CSS



CSS can be added to HTML elements in 3 ways:

1.Inline - by using the **style** attribute in HTML elements
Applied to only one element

2. Internal - by using a **<style>** element in the **<head>** section
Applied to entire web page

3. External - by using an **external .CSS file**

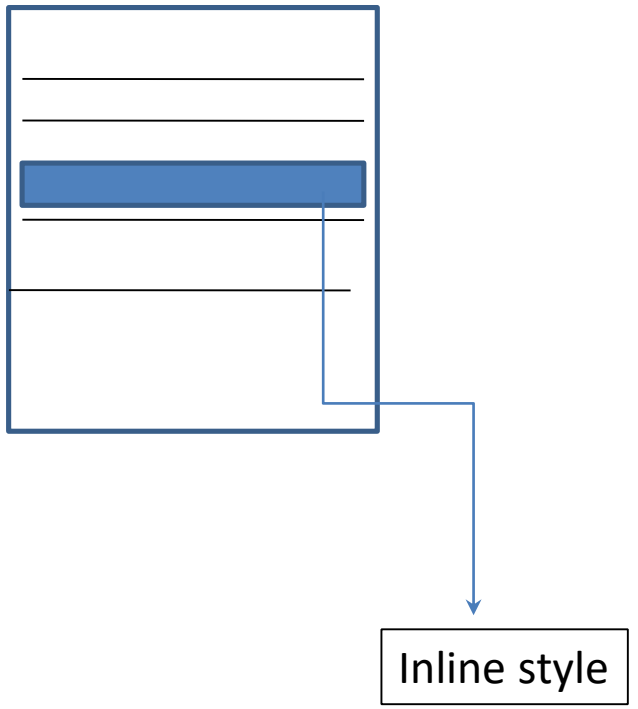
CSS- Cascading Style Sheets

1. Inline CSS

- Inline CSS is used to apply CSS on a **single line or element.**
- An inline CSS uses the **style** attribute of an HTML element.

```
<h1 style="color:blue;">This is a Blue Heading</h1>
```

This example sets the **text color** of the < h1> element to blue:




```
<html>
<body>

<h1 style="color:blue;">This is a Blue Heading</h1>

</body>
</html>
```



CSS- Cascading Style Sheets

CHITKARA UNIVERSITY

1. Inline CSS

Style is applicable to only one tag

```
<html>
<body>
<h1 style="color:red;margin-left:40px;">Inline CSS is applied on
this heading.</h1>
<p>This paragraph is not affected.</p>
</body>
</html>
```

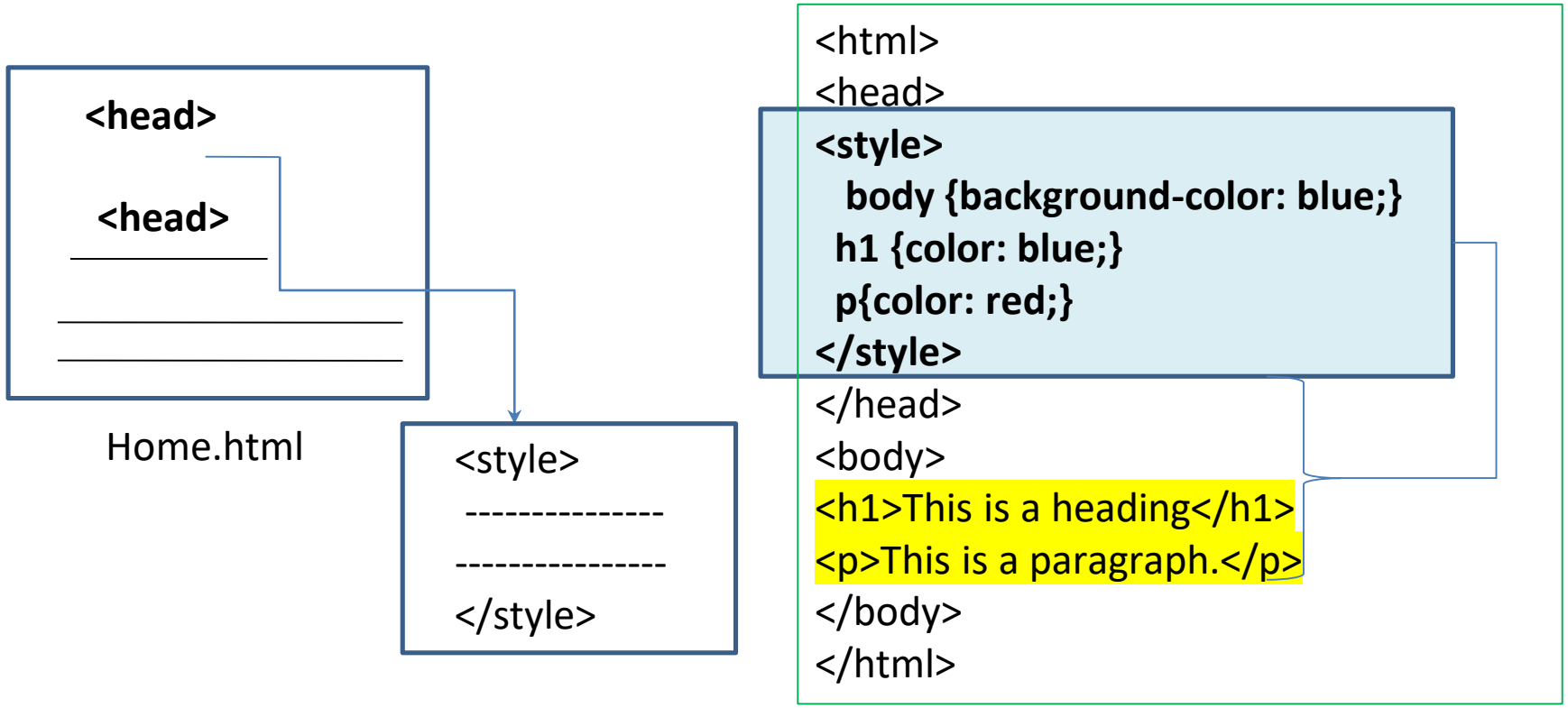
Inline CSS is applied on this heading.

This paragraph is not affected.

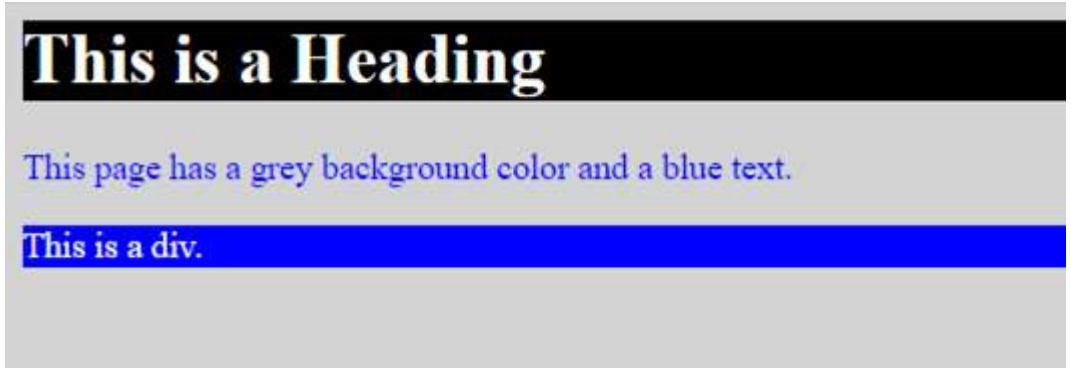
CSS- Cascading Style Sheets

2. Internal CSS: An internal CSS is used to define a **style for a single HTML page**.

The internal style is defined inside the **<style>** element, inside the **<head>** section.



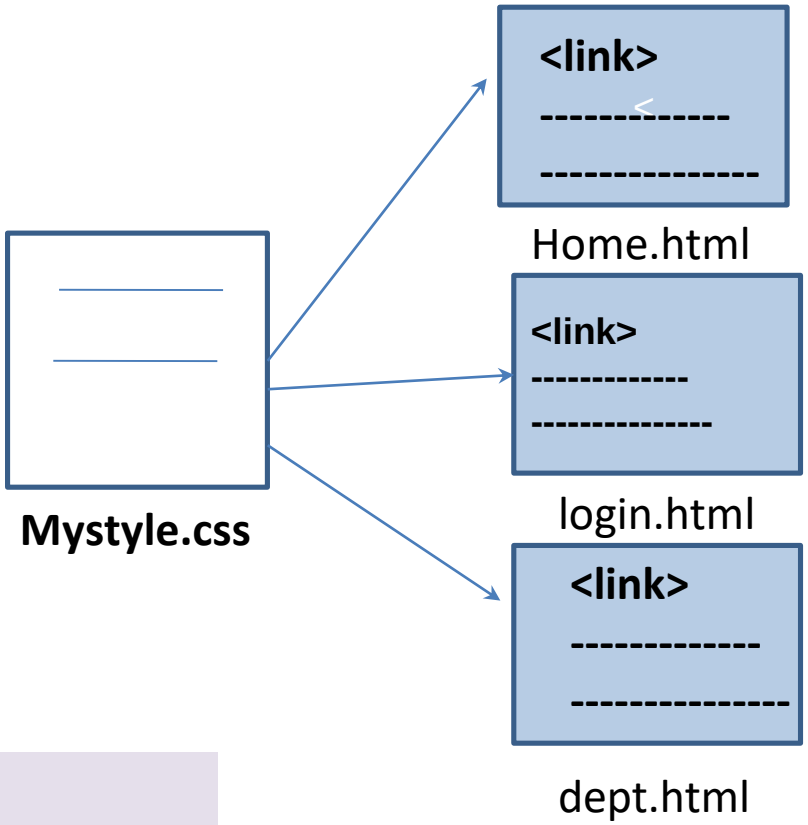
2. Internal CSS




```
<html>
<head>
 <style>
                body {
                     background-color: lightgrey;
                    color: blue;
                    }
                 h1 {
                    background-color: black;
                      color: white;
                      }
                 div {
                      background-color: blue;
                      color: white;
                      }
</style>
</head>
<body>
<h1>This is a Heading</h1>
<p>This page has a grey background color and a blue text.</p>
<div>This is a div.</div>
</body>
</html>
```

3. External CSS:-

- An external style sheet is used to **define the style for many HTML pages.**
- With an external style sheet, we can **change the look of an entire web site**, by changing **one file**
- An External style sheet is written in **separate file with extension .css** and **referenced in multiple HTML documents**
- To use an **external style sheet**, add a **<link>** in the <head> section of the HTML page:



Syntax to Link external CSS file:

```
<link rel="stylesheet" type="text/css" href="mystyle.css">
```

CSS- Cascading Style Sheets

- An external style sheet can be written in **any text editor,** and must be saved with a **.css extension**.
- The external **.css file** should not contain any HTML tags.

```
body
{background-color:lightblue;
}
h1{
    color:navy;
    margin-left:20px;
}
```

"mystyle.css“

```
<html>
<head>
        <link rel="stylesheet" type="text/css"
href="mystyle.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

“home.html“

Attributes of <link> tag:

1. The **“rel”** attribute is **compulsory** and it is used to **specify the relationship** between the current file and the linked file.
2. The **“href”** □ it is **compulsory** used to **specify the file** location.
3. The “**type”** □ attribute is **optional**, it is used to **define the type of content** that we are linking.

CSS SELECTORS

- **A CSS selector selects the HTML element(s) you want to style.**

▪There are several different types of selectors in CSS.

1.Element Selector(element name)
2.Id Selector(#)
3.Class Selector(.)
4.Universal Selector(*)
5.Group Selector(h1,p,.)

1) CSS Element Selector

- The element selector selects the HTML element by name.
- The style is applicable to the tags that are same .

```
<!– ELEMENT SELECTOR-->
<html>
<head>
<style>
  p{
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<p>This style will be applied on every paragraph.</p>
<p >here also applied</p>
<p>And here</p>
</body>
</html>
```

CSS SELECTORS

CHITKARA UNIVERSITY

2) CSS Id Selector(#)

- The **id** selector selects the **id** attribute of an HTML element to select a specific element.
- An id is always unique within the page so it is chosen to select a single, unique element.
- It is written with the **hash character (#),** followed by the id of the element.

```
<html>
<head>
<style>
#para1 {
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<p id="para1">applicable to this element only</p>
<p>This paragraph will not be affected.</p>
</body>
</html>
```

CSS SELECTORS

CHITKARA UNIVERSITY

3) CSS Class Selector

- The class selector **selects HTML elements with a specific class attribute.**
- It is used with a **period character** . (full stop symbol) followed by the class name.
- A class name should not be started with a number.

CSS Class Selector for specific element

If you want to **specify that only one specific HTML element should be affected**, then you should use the element name with class selector.

```
<html>
<head>
<style>
.center {
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<h1 class="center">This heading is blue and center-aligned.</h1>
<p class="center">This paragraph is blue and center-aligned.</p>
</body>
</html>
```

```
<html>
<head>
<style>
p.center {
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<h1 class="center">This heading is not affected</h1>
<p class="center">This  is blue and center-aligned.</p>
</body>
</html
```

CSS SELECTORS

4) CSS Universal Selector (*)

- It selects all the elements on the page.

```
<html>
<head>
<style>
* {
   color: green;
   font-size: 20px;
   }
</style>
</head>
<body>
<h2>This is heading</h2>
<p>This style will be applied on every paragraph.</p>
<p id="para1">Me too!</p>
<p>And me!</p>
</body>
</html>
```

CSS SELECTORS

CHITKARA UNIVERSITY

5) CSS Group Selector:

- The grouping selector is used to select all the elements with the same style definitions.
- Grouping selector is used to minimize the code. Commas are used to separate each selector in grouping.

```
<html>
<head>
<style>
h1, h2, p {
            text-align: center;
             color: blue;
           }
</style>
</head>
<body>
<h1>Hello..This is heading one /h1>
<h2>Hello   this is  heading two </h2>
<p>This is a paragraph.</p>
</body>
</html>
```

All CSS Simple Selectors

CHITKARA UNIVERSITY

Selector	Example	Example description
#id	#firstname	Selects the element with id="firstname"
.class	.intro	Selects all elements with class="intro"
*	*	Selects all elements
element	p	Selects all <p> elements
element,element,..	div, p	Selects all elements and all <p> elements

CSS- Multiple Classes

- Multiple classes can be applied to a single HTML element by **adding each class name** to the class attribute, **separated by a space.**

```
<style>
   .class_1{
         /* some styles */
       }

   .class_2{
      /* some styles */
   }
</style>

Here two classes are defined
```

```
<tag_name class="class_1 class_2">

Here the two class style are applied to tag
```

```
<style>
        .para1 {
                font-size: larger;
                margin-bottom: 35px;
                background-color: lightgreen;
              }
        .para2 {
                color: red;
               }
</style>
<body>
        <p class="para1"> only one style is applied </p>
        <p class="para1 para2">Two classes styles are applied
        </p>

</body>
```

SPECIFICITY IN CSS

- Specificity in CSS is decides which styles to apply when multiple styles are defined for the same element
- i.e. if there are two or more CSS styles applied to the same element, then the selector with the **highest specificity value** **will be applied to that HTML element.**
- **[DIFFERENT SELECTORS are id selector(#), class selector(.), element selector etc.]**




```
<html>
<head>
<style>

    .test {color: green;}   /*class selector*/

    p {color: red;}         /*element selector */

</style>
</head>
<body>

<p class="test">Hello World!</p>
                    <!- - here the class selector is
applied -- >
</body>
</html>
```

SPECIFICITY FOR DIFFERENT SELECTORS

CHITKARA UNIVERSITY

//ID SELECTOR IS GIVEN HIGHER PRIORITY in [ID, CLASS, ELEMENT]

Ex1:

```
<html>
<head>
<style>
#demo {color: blue;}  /* id selector */
.test {color: green;}     /* class selector */
p {color: red;}           /* element selector*/
</style>
</head>
<body>
<p id="demo" class="test">Hello World!</p>

</body>
</html>
```

- Here **ID selector is given higher priority, so Hello World will be displayed in Blue**

//INLINE STYLE IS GIVEN THE HIGHEST PRIORITY IN [ID ,CLASS, ELEMENT, INLINE}

Ex 2.

```
<html>
<head>
<style>
#demo {color: blue;} /* id selector */
.test {color: green;}  /* class selector */
 p {color: red;}       /* element selector*/
</style>
</head>
<body>
<p id="demo" class="test" style="color: pink;">Inline style</p>
                                        <! - - inline selector  is appled-->
</body>
</html>
```

- Here we have added an inline style for the "p" element.
- The text will now be pink, because the **inline style is given the highest priority**

Specificity hierarchy

- Each selector has a place in the hierarchy.

FOUR categories define the selector's specificity level:

PRIORITY 1: INLINE STYLES (highest priority)

Example: **<h1 style="color: pink;">**

PROORITY 2: ID SELECTOR (#)

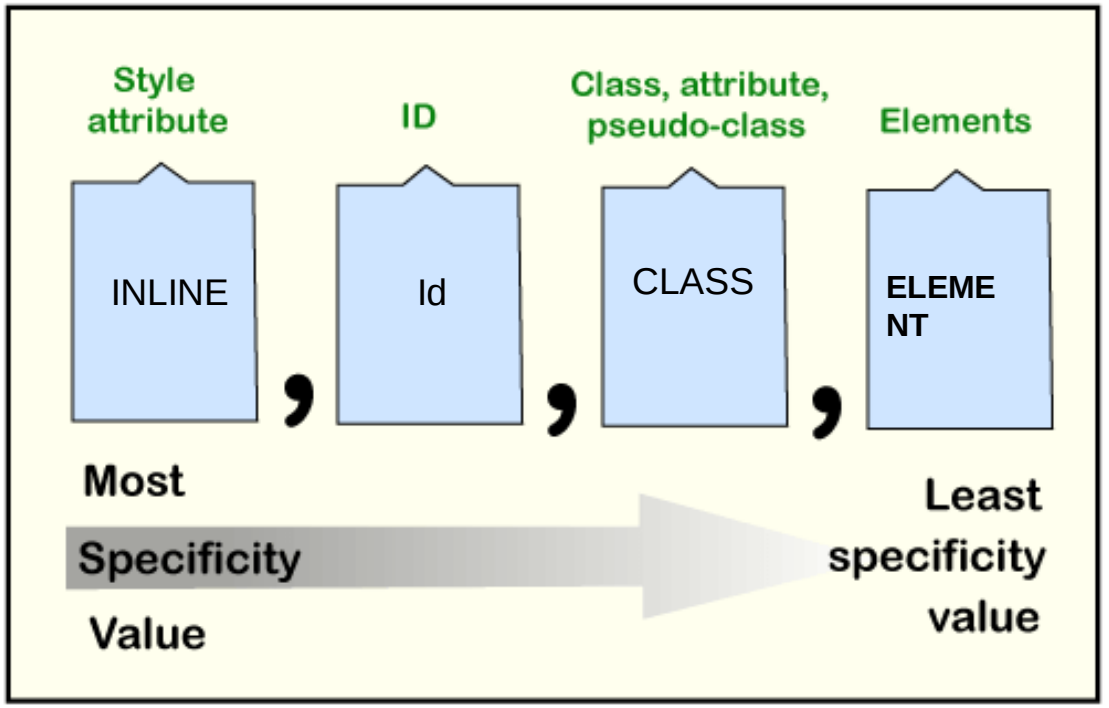
- Example: **#navbar**

PROIRITY 3: CLASS SELECTOR, pseudo-classes, attribute selectors -

Example: **.test,** :hover, [href]

PRIORITY 4 ELEMENT SELECTOR and pseudo-elements -

Example: **h1, ::before**



CSS Pseudo-classes

- A pseudo-class is used to define a **special state of an element.**

For example, it can be used to:

- Style an element when a user mouses over it
- Style visited and unvisited links differently
- Style an element when it gets focus

```
selector:pseudo-class {
  property: value;
}
```

```
/* unvisited link */
a:link {
  color: #FF0000;
}

/* visited link */
a:visited {
  color: #00FF00;
}

/* mouse over link */
a:hover {
  color: #FF00FF;
}

/* selected link */
a:active {
  color: #0000FF;
}
/*select element to get focus */
a: focus{outline:dashed;
      background-color: grey;}
```

Specificity rules

- **Equal specificity: the latest rule wins - If the same rule is written twice, then the latest(last) rule will be applied.**

```
h1 {background-color: yellow;}
h1 {background-color: red;}   /* highest priority */
```

- The specificity of **class selector is greater than the element selectors**

```
<style>
.intro {background-color: yellow;}
h1 {background-color: red;}
</style>
<body>
<h1 class="intro">This is a heading</h1>
</body>
```

CSS Combinators

- **A combinator is something that explains the relationship between the selectors.**

FOUR DIFFERENT COMBINATORS IN CSS:

1. Descendant selector (space)

div p □ <p> is descendant of

2. child selector (>)

**div > p --> <p> is child of **

3. adjacent sibling selector (+)

**div + p ---> <p> is adjacent sibling of **

4. general sibling selector (~)

div ~ p --> <p> is general sibling selector

```
<div>      <!-- Descendant selector-- >
  <p>Para 1 in the div.  It is descendant  of div </p>
  <p>Para 2 in the div. It is descendant  of div </p>
   <section>     <p>Para3 in the div. it  is descendant</p>     </section>
</div>
```

```
<div>              <!-- child selector -->
  <p>Para 1 in the div. it is child of div</p>  □  CHILD OF DIV
  <p>Para 2 in the div. it is child of div </p> □  CHILD OF DIV
   <section>     <p>Para 3 in the div. but not a child</p>     </section>
</div>
```

```
<div>  <!-- adjacent sibling selector -->
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
</div>
<p>Para 3. After a div. it is adjacent sibling</p>  □  ADJUSENT SIBLING
```

```
<div>   <p>Paragraph 2.</p> </div>     <!-- general sibling selector -->
<p>Paragraph 3.</p>     □     GENERAL SIBLING
    <code>Some code.</code>
<p>Paragraph 4.</p>  □  GENERAL SIBLINE
```

CSS Combinators

1.DESCENDANT SELECTOR(SPACE)

- The descendant selector **matches all elements that are descendants** of a specified element.
- The example selects all <p> elements inside elements:

```
<!– DESCENDANT SELECTOR -- >
<html>
<head>
<style>
div p        /* p is descendant of div*/
 {
  background-color: yellow;
}
</style>
</head>
<body>
<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section><p>Paragraph 3 in the div.</p></section>
</div>
</body>
</html?
```

CSS Combinators

2. Child Selector (>)

- The child selector **selects all elements that are the children** of a specified element.
- The example selects all **<p>** elements that are **children of a ** element:

Example

```
div > p
{
  background-color: yellow;
}
```

```
<style>
div > p      /* p is child  of div*/
{
  background-color: yellow;
}
</style>
.
.
<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section>    <!-- not Child but Descendant -->
    <p>Paragraph 3 in the div (inside a section element).</p>
  </section>
  <p>Paragraph 4 in the div.</p>
</div>
```

CSS Combinators

3. Adjacent Sibling Selector (+)

- ☐ used to select an element that **is directly after another specific element.**
- Sibling elements must have the same parent element, and "adjacent" means "immediately following".

```
<style>
div + p    /* p is adjacent sibling of div*/
{

  background-color: yellow;
}
</style>
```

```
..
<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
</div>
<p>Paragraph 3. After a div.</p>
<p>Paragraph 4. After a div.</p>
<div>
  <p>Paragraph 5 in the div.</p>
  <p>Paragraph 6 in the div.</p>
</div>

<p>Paragraph 7. After a div.</p>
<p>Paragraph 8. After a div.</p>

  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section>    <!-- not Child but Descendant -->
    <p>Paragraph 3 in the div (inside a section element).</p>
  </section>
  <p>Paragraph 4 in the div.</p>
</div>
```

CSS chaining Selectors

- Chaining selectors is a technique in CSS where **multiple selectors are combined to apply styles to elements** that **meet all the specified criteria.**

- Chaining selectors uses the combinators.

```
.container > p      /* chaining selector */
 {
  font-size: 24px;
}

<div class="container">

  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
</div>
```

Ex.

.container > p ,

Applies on the **p** element that is a **direct child** of an element with **class "container"**.

- It applies a font size of 24px to the **p** element.

CSS chaining Selectors

- **.container p + ul li**,

Applies style on the

1. <**li> elements that are direct children of a **
2. **<uL> element** that **is adjacent sibling of <p> element**
3. **<P>** that is a **descendant of an element with class = "container**".

- It applies a red color to these li elements.

```
.container p + ul li
 {
  color: red;
}
```

```
<div class="container">
  <h1>Title</h1>
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
  <ul>
    <li>List item 1</li>
    <li>List item 2</li>
  </ul>
</div>
```

CSS chaining Selectors

- Chaining selectors can also be used with classes, IDs, and attribute selectors to target specific elements with a particular set of properties.

- .container .special
- we first specify the parent element with a class of **"container",** then we specify the child element with a class of **"special“**
- **The style is applied on Paragraph2**

```
.container .special
 {
  font-weight: bold;
  color: blue;
}

<div class="container">
  <h1>Title</h1>
  <p>Paragraph 1</p>
    <p class="special">Paragraph 2</p>
</div>
```

Nested Elements in css

CHITKARA UNIVERSITY

- In CSS, nested elements refer to the **use of selectors** that are **nested within other selectors** to target specific elements within the HTML document.

- Selectors can be nested using the **parent-child relationship.**
- For example, you can target all **<li>** elements that are children of a **ul** element using the following selector:
- **ul li**

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>
    Item 3
    <ul>
      <li>Subitem 1</li>
      <li>Subitem 2</li>
    </ul>
  </li>
</ul>
```

```
ul li ul li {
  color: blue;
}
```

CSS Units

CSS units are used to define the length and size of several properties.
Several CSS properties take "length" values, such as `font-size`, `width`, `margin`, `padding`, `border`, etc.
CSS has two types of units:

- Absolute units
- Relative units

CSS Units

CSS Absolute Units

- Absolute units are fixed, and the length expressed in any of these will appear exactly that size.

- Absolute units do NOT change when the screen size change, and are not recommended for websites. However, they can be used if the output medium is known, such as for print layout.

- The most used absolute unit is px (pixels).

Absolute Type

- px
- cm
- mm

CSS Units

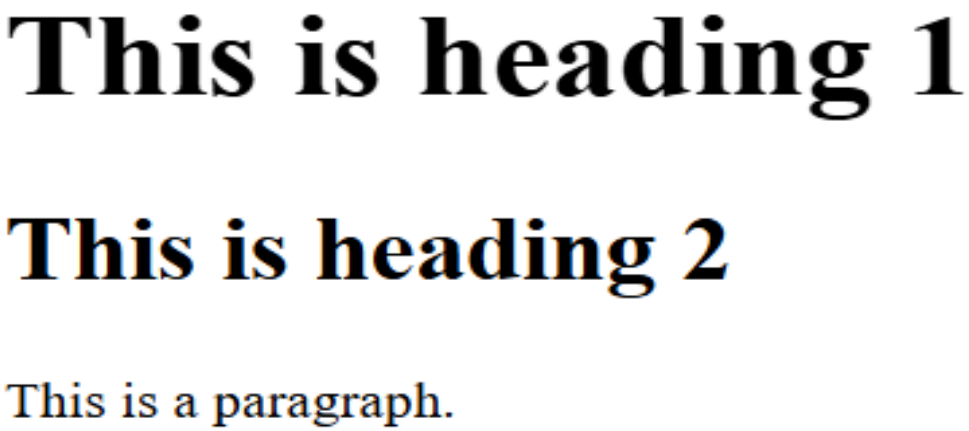
```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
  font-size: 40px;
}

h2 {
  font-size: 30px;
}

p {
  font-size: 17px;
}
</style>
</head>
<body>

<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<p>This is a paragraph.</p>

</body>
</html>
```



CSS Relative Units

- Relative units specify a length relative to another length property (like parent element, root element, or viewport).
- Relative length units scale better between different screen sizes, making them ideal for responsive web design.
- The two most used relative units are em and rem.

Relative Type

- em
- rem
- %
- vw
- vh

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  font-size: 16px; /* Base font size */
}

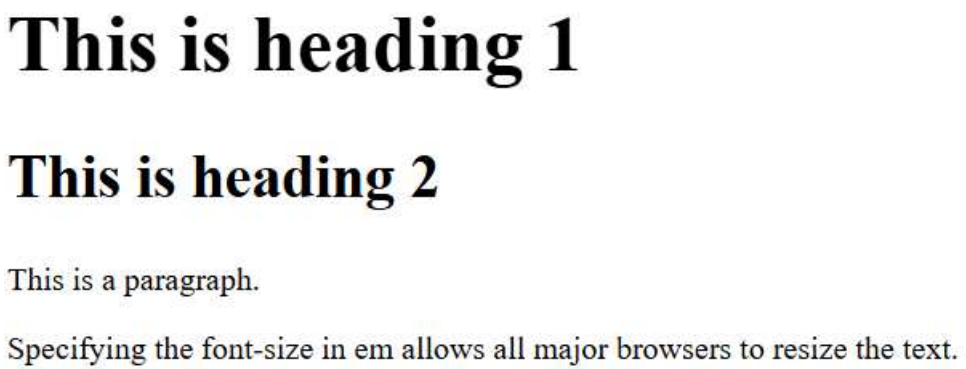
h1 {
  font-size: 2.5em; /* 2.5 * 16 = 40px */
}

h2 {
  font-size: 1.875em; /* 1.875 * 16 = 30px */
}

p {
  font-size: 1em; /* 1 * 16 = 16px */
}
</style>
</head>
<body>

<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<p>This is a paragraph.</p>
<p>Specifying the font-size in em allows all major browsers to resize the text.</p>

</body>
</html>
```



Colors in CSS

In CSS, colors are specified by using a predefined color name, or with a RGB, HEX, HSL, RGBA, HSLA value.

CSS Color Names

In CSS, a color can be specified by using a predefined color name:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="background-color:Tomato;">Tomato</h1>
<h1 style="background-color:Orange;">Orange</h1>
<h1 style="background-
color:DodgerBlue;">DodgerBlue</h1>
<h1 style="background-
color:MediumSeaGreen;">MediumSeaGreen</h1>
<h1 style="background-color:Gray;">Gray</h1>
<h1 style="background-color:SlateBlue;">SlateBlue</h1>
<h1 style="background-color:Violet;">Violet</h1>
<h1 style="background-color:LightGray;">LightGray</h1>

</body>
</html>
```

Tomato

Orange

DodgerBlue

MediumSeaGreen

Gray

SlateBlue

Violet

LightGray

Colors in CSS

CSS Background Color

You can set the background color for HTML elements:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="background-color:DodgerBlue;">Hello
World</h1>

<p style="background-color:Tomato;">
Lorem ipsum dolor sit amet, consectetuer adipiscing
elit, sed diam nonummy nibh euismod tincidunt ut
laoreet dolore magna aliquam erat volutpat.
Ut wisi enim ad minim veniam, quis nostrud exerci
tation ullamcorper suscipit lobortis nisl ut aliquip ex ea
commodo consequat.
</p>

</body>
</html>
```



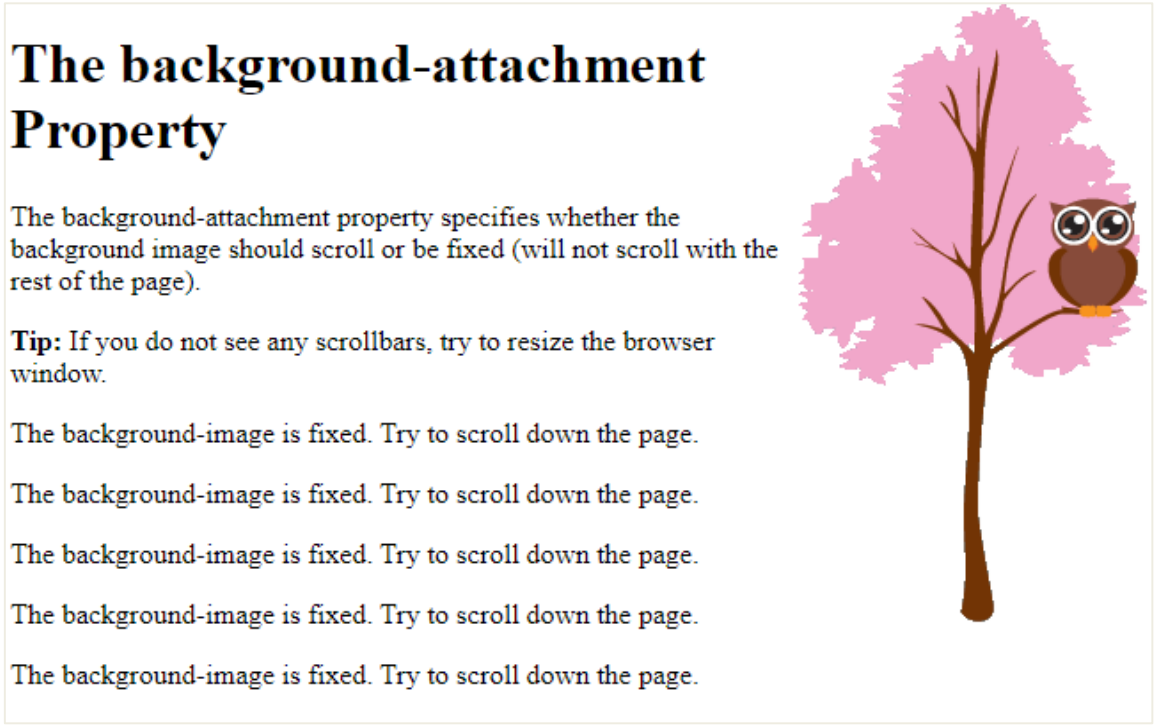
CSS Background

- CSS background property is used to define the background effects on element.

4. **background-attachment** specifies whether the background image should scroll or be fixed

```
body {
  background-image: url("img_tree.png");
  background-repeat: no-repeat;
  background-position: right top;
  margin-right: 200px;
  background-attachment: fixed;
}
```

5. **background-position** specifies initial position of the background image
   - center
   - top
   - bottom
   - left
   - right



CSS Background

- CSS background property is used to define the background effects on element.

CSS background properties: 5 BG properties

1. background-color

```
background-color: lightblue;
```

Opacity / Transparency

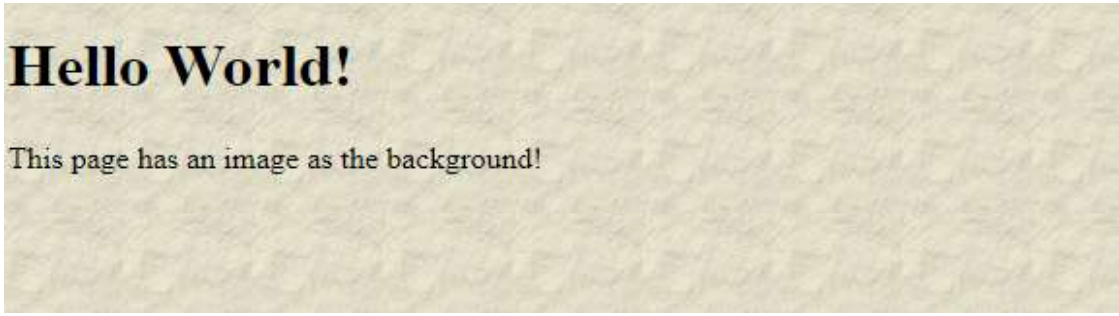
```
div {
  background-color: green;
  opacity: 0.3;
}
```



2 . background-image:

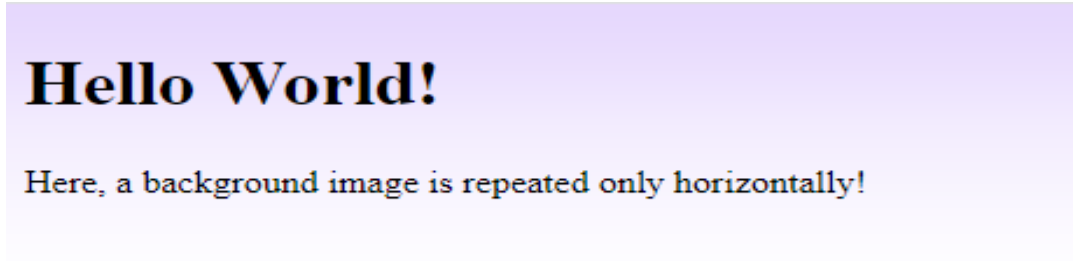
- specifies an image to use as the background of an element
- By default, the image is repeated so it covers the entire element.

```
body {
  background-image: url("paper.gif");
}
```



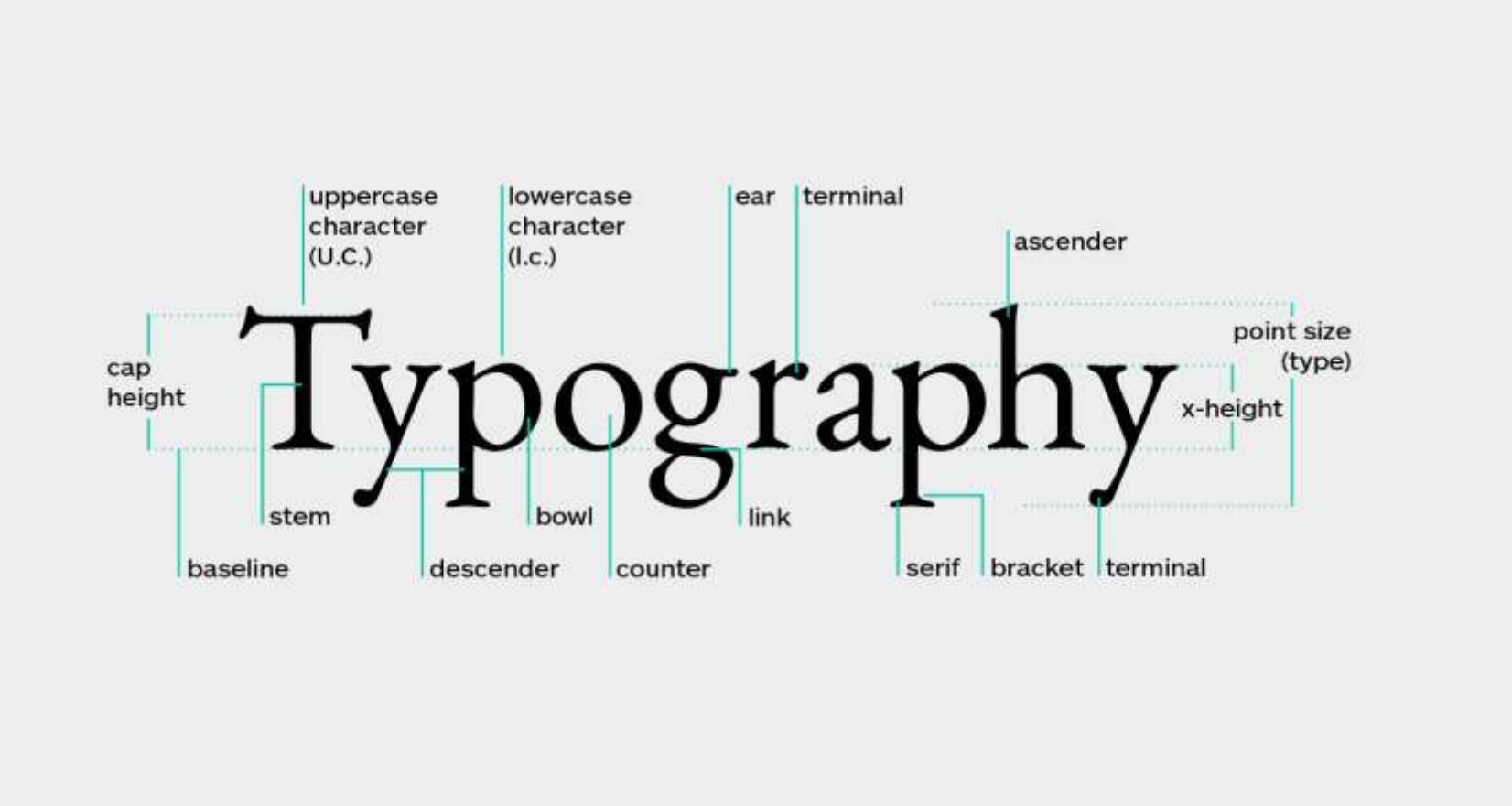
3. . background-repeat: repeats the background image horizontally and vertically.

```
body {          background-image: url("gradient_bg.png");
background-repeat: repeat-x;      }
```



Typography

- **Typography** □ arrangement of text **to make written language legible, readable and appealing when displayed**.
- The arrangement of type involves selecting **typefaces, point sizes, line lengths, line-spacing, and letter-spacing**, as well as **adjusting the space between pairs of letters.**



CSS Fonts

CHITKARA UNIVERSITY

In CSS there are FIVE generic font families:

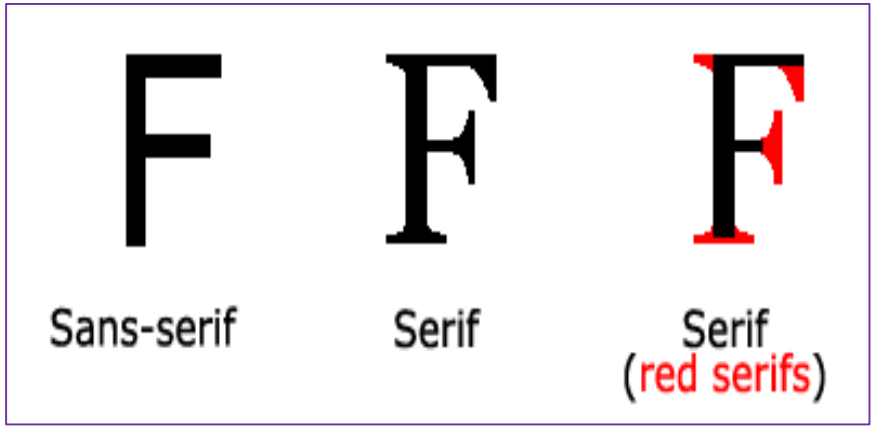
1) **Serif** fonts have a **small stroke at the edges of each letter.** They create a sense of formality and elegance.

2) Sans-serif fonts have **clean lines** (no small strokes attached). They create a modern and minimalistic look.

3) Monospace fonts - here all the letters have the same fixed width. They create a mechanical look.

4) Cursive fonts imitate human handwriting.

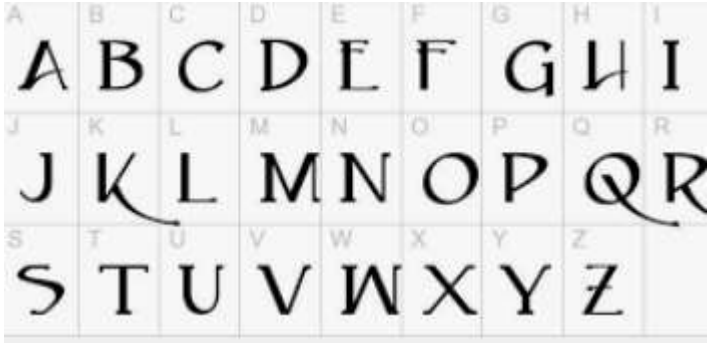
5)Fantasy fonts are decorative/playful fonts.

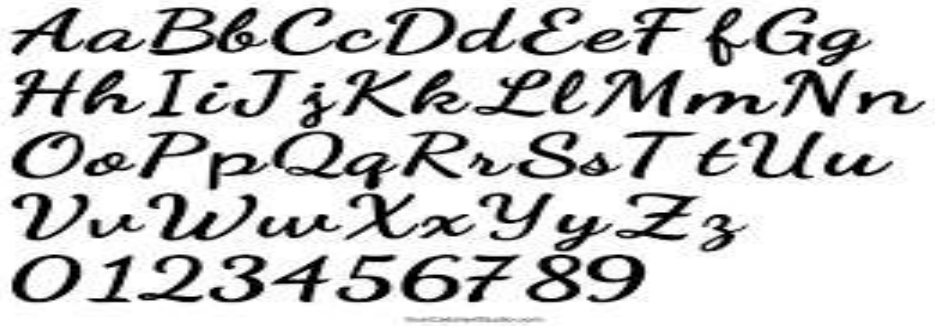


monospace

Ubuntu monospace

monospace





CSS Fonts

CHITKARA UNIVERSITY

Generic Font Family	Examples of Font Names
Serif	Times New Roman Georgia Garamond
Sans-serif	Arial Verdana Helvetica
Monospace	Courier New Lucida Console Monaco
Cursive	*Brush Script MT* *Lucida Handwriting*
Fantasy	Copperplate *Papyrus*

CSS Fonts font-family

The CSS font-family Property

In CSS, we use the font-family property to specify the font of a text.

```
.p1 {
      font-family: "Times New Roman", Times, serif;
      }

.p2 {
  font-family: Arial, Helvetica, sans-serif;
}

.p3 {
  font-family: "Lucida Console", "Courier New", monospace;
}
</style>
```

CSS font-family

This is a paragraph, shown in the Times New Roman font.

This is a paragraph, shown in the Arial font.

This is a paragraph, shown in the Lucida Console font.

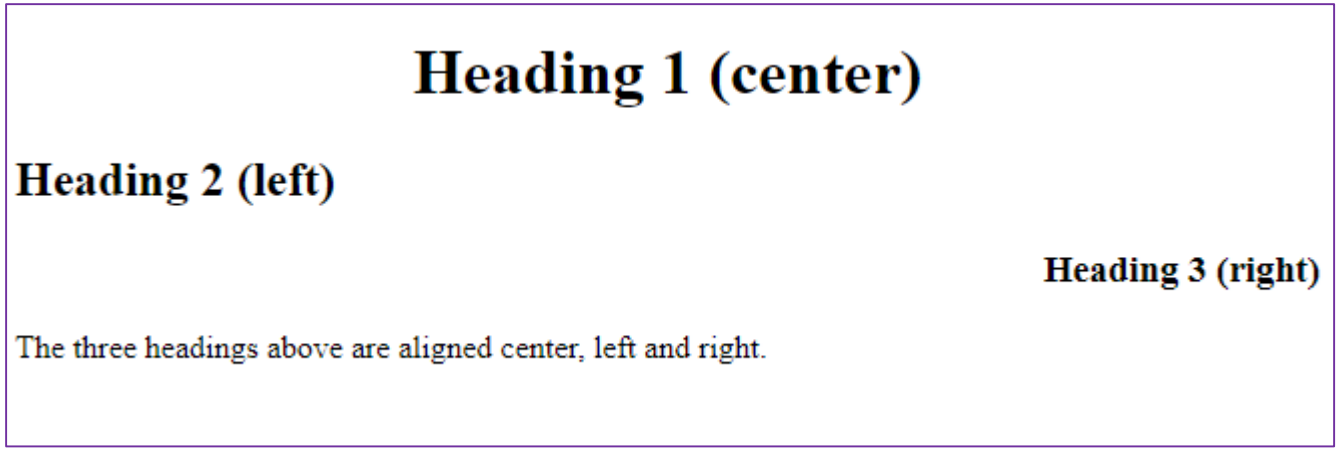
- "**Arial**" is a sans-serif **font-family name**
- "**Helvetica**" is a similar sans-serif font used as a **fallback option** in case "Arial" is not available on a particular system;
- "**Sans-serif**" is a generic font family that includes all sans-serif fonts.

Text Alignment

```
<html>
<head>
<style>
h1 {
  text-align: center;
}
h2 {
  text-align: left;
}
h3 {
  text-align: right;
}
</style>
</head>
<body>
<h1>Heading 1 (center)</h1>
<h2>Heading 2 (left)</h2>
<h3>Heading 3 (right)</h3>
<p>The three headings above are aligned center,
left and right.</p>
</body>
</html>
```

The text-align property is used to set the horizontal alignment of a text.

A text can be left or right aligned, centered, or justified.



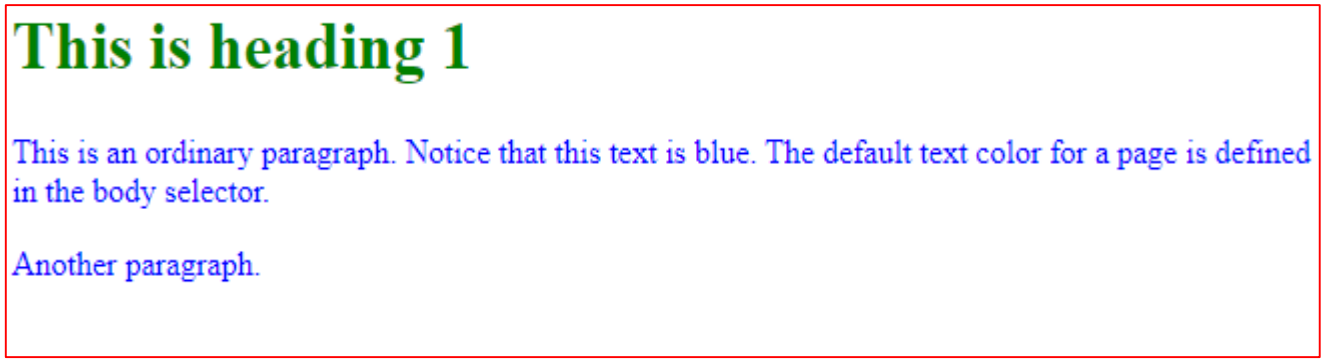
Text Color

The color property **is used to set the color of the text. The color is specified by:**

a color name - like "red"
a HEX value - like "#ff0000"
an RGB value - like "rgb(255,0,0)"

```
<html>
<head>
<style>
body {
  color: blue;
}
h1 {
  color: green;
}
</style>
</head>
<body>

<h1>This is heading 1</h1>
<p>This is an ordinary paragraph. Notice that this text is blue.
The default text color for a page is defined in the body
selector.</p>
<p>Another paragraph.</p>
</body>
</html>
```



CSS Backgrounds :background-color

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: lightblue;
}
</style>
</head>
<body>

<h1>Hello World!</h1>

<p>This page has a light blue background color!</p>

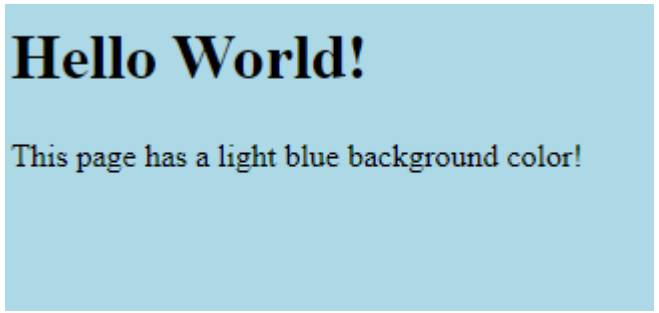
</body>
</html>
```

CSS background-color

- The background-color property specifies the background color of an element.

With CSS, a color is most often specified by:

- a valid color name - like "red"
- a HEX value - like "#ff0000"
- an RGB value - like "rgb(255,0,0)"



CSS Background Image

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-image: url("paper.gif");
}
</style>
</head>
<body>

<h1>Hello World!</h1>

<p>This page has an image as the background!</p>

</body>
</html>
```

The background-image property specifies an image to use as the background of an element.

By default, the image is repeated so it covers the entire element.



CSS Opacity / Transparency

- The opacity property specifies the opacity/transparency of an element.
- The `opacity` property can take a value from 0.0 - 1.0. The lower the value, the more transparent:

```
<html>
<head>
<style>
img {
  opacity: 0.5;
}
</style>
</head>
<body>

<h1>Image Transparency</h1>
<p>The opacity property specifies the transparency of an element. The lower the value, the more transparent:</p>

<p>Image with 50% opacity:</p>
<img src="img_forest.jpg" alt="Forest" width="170" height="100">

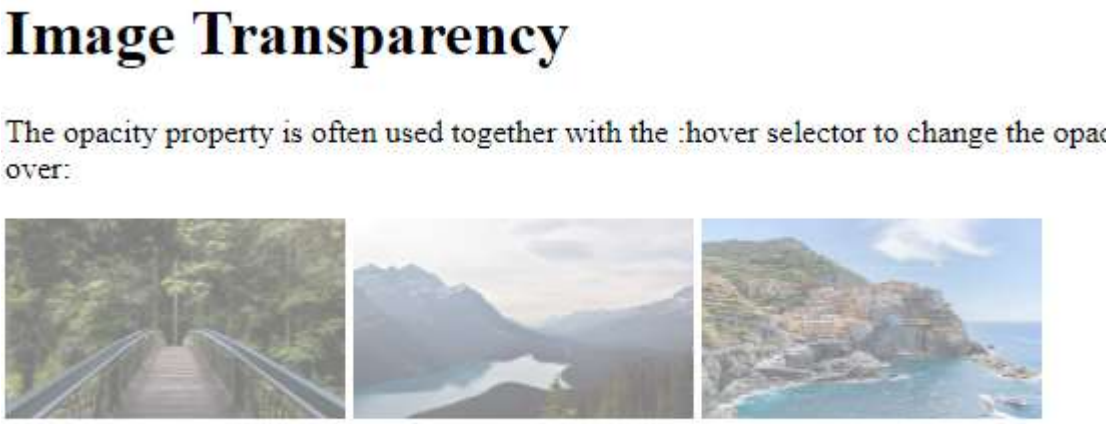
</body>
</html>
```



Transparent Hover Effect

```
<html>
<head>
<style>
img {
  opacity: 0.5;
}
img:hover {
  opacity: 1.0;
}
</style>
</head>
<body>
<h1>Image Transparency</h1>
<p>The opacity property is often used together with the :hover
selector to change the opacity on mouse-over:</p>
<img src="img_forest.jpg" alt="Forest" width="170" height="100">
<img src="img_mountains.jpg" alt="Mountains" width="170"
height="100">
<img src="img_5terre.jpg" alt="Italy" width="170" height="100">
</body>
</html>
```

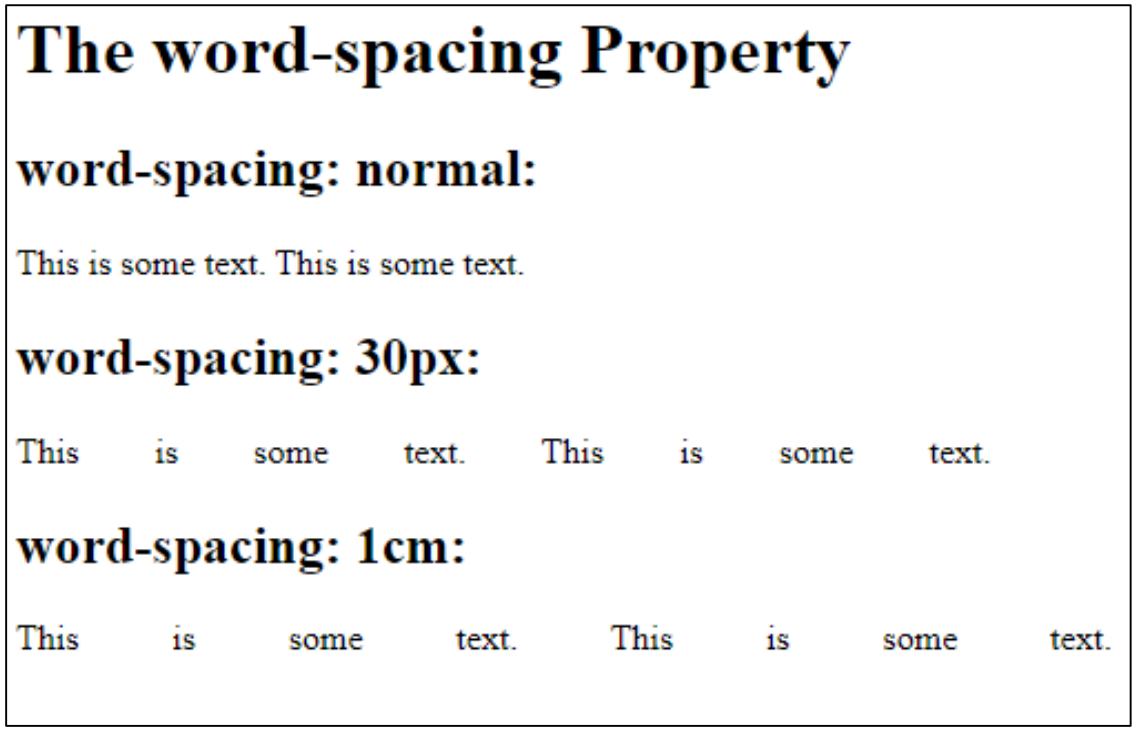
The `opacity` property is often used together with the `:hover` selector to change the opacity on mouse-over:



Word Spacing

```
<html>
<head>
<style>
p.a {
  word-spacing: normal;
}
p.b {
  word-spacing: 30px;
}
p.c {
  word-spacing: 1cm;
}
</style>
</head>
<body>
<h1>The word-spacing Property</h1>
<h2>word-spacing: normal:</h2>
<p class="a">This is some text. This is some text.</p>
<h2>word-spacing: 30px:</h2>
<p class="b">This is some text. This is some text.</p>
<h2>word-spacing: 1cm:</h2>
<p class="c">This is some text. This is some text.</p>
</body>
</html>
```

The `word-spacing` property **increases or decreases** the **white space** between words.



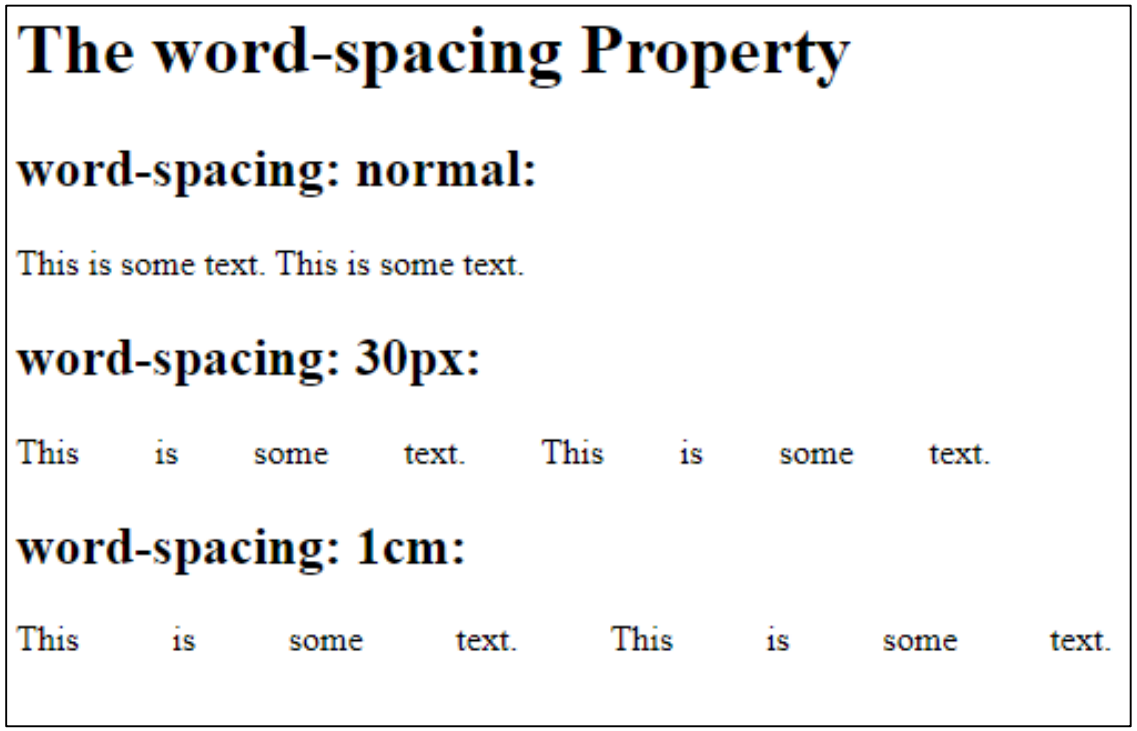
LETTER-SPACING

```
<html>
<head>
<style>
h1 {
  letter-spacing: 3px;
}

h2 {
  letter-spacing: 2px;
}

h3 {
  letter-spacing: -1px;
}
</style>
</head>
<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
</body>
```

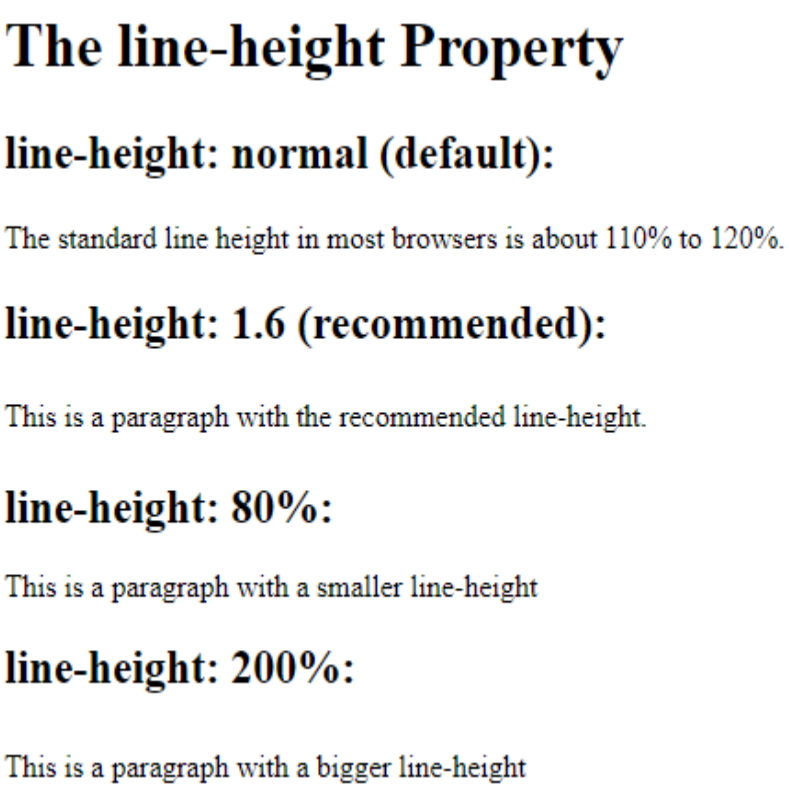
The `letter-spacing` property increases or decreases the space between characters in a text.



CSS line-height Property

The `line-height` property specifies the height of a line.

```
<html>
<head>
<style>
div.a {
  line-height: normal;
}
div.b {
  line-height: 1.6;
}
div.c {
  line-height: 80%;
}
div.d {
  line-height: 200%;
}
</style>
</head>
<body>
<h2>line-height: normal (default):</h2>
<div class="a">
The standard line height in most browsers is about 110% to 120%.</div>
<h2>line-height: 1.6 (recommended):</h2>
<div class="b">This is a paragraph with the recommended line-height.<br>
</div>
<h2>line-height: 80%:</h2>
<div class="c">This is a paragraph with a smaller line-height</div>
<h2>line-height: 200%:</h2>
<div class="d">This is a paragraph with a bigger line-height</div>
</body>
</html>
```



CSS @font-face Rule

```
<html>
<head>
<style>
@font-face {
  font-family: myFirstFont;
  src: url(sansation_light.woff);
}

div {
  font-family: myFirstFont;
}
</style>
</head>
<body>

<h1>The @font-face Rule</h1>

<div>With CSS, websites can finally use fonts other than the pre
selected "web-safe" fonts.</div>

</body>
</html>
```

- With the @font-face rule, web designers do not have to use one of the "web-safe" fonts anymore.
- In the @font-face rule you must first define a name for the font (e.g. myFirstFont), and then point to the font file.
- To use the font for an HTML element, refer to the name of the font (myFirstFont) through the font-family property:

Linking Fonts

```
<html>
<head>
<style>
@font-face {
   font-family: myFirstFont;
   src: url(sansation_light.woff);
}
* {
   font-family: myFirstFont;
}
</style>
</head>
<body>
<h1>The @font-face Rule</h1>
<div>
With CSS, websites can use fonts other than the pre-selected
"web-safe" fonts.
</div>

</body>
</html>
```

The CSS @font-face used to link external fonts
Web fonts allow Web designers to use fonts that are not installed on the user's computer.

CSS @font-face Rule

```
<html>
<head>
<style>
@font-face {
  font-family: myFirstFont;
  src: url(sansation_light.woff);
}

div {
  font-family: myFirstFont;
}
</style>
</head>
<body>

<h1>The @font-face Rule</h1>

<div>With CSS, websites can finally use fonts other than the pre
selected "web-safe" fonts.</div>

</body>
</html>
```

- With the @font-face rule, web designers do not have to use one of the "web-safe" fonts anymore.
- In the @font-face rule you must first define a name for the font (e.g. myFirstFont), and then point to the font file.
- To use the font for an HTML element, refer to the name of the font (myFirstFont) through the font-family property:

CSS Gradients

The CSS gradient functions let you display smooth transitions between two or more colors within an element.

CSS defines three types of gradients:

- **Linear Gradients** - The color transition goes down, up, left, right, or diagonally
- **Radial Gradients** - The color transition goes out from a central point
- **Conic Gradients** - The color transition is rotated around a center point

The CSS gradient functions are used within the `background-image` property.



CSS linear-gradient() Function

- The CSS `linear-gradient()` function creates a linear gradient.
- A linear gradient defines a color transition that goes in a straight line, it can go down, up, to left, to right, or diagonally.
- A linear gradient requires at least two color stops. Color stops are the colors you want to render smooth transitions among. You can also set a starting point and a direction (or an angle) along with the gradient effect.

Syntax

```
background-image: linear-gradient(direction, color-stop1, color-stop2, ...);
```

Direction – Top to Bottom (this is default)

The following example shows a linear gradient that goes from top to bottom. It starts red, transitioning to yellow:

```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: linear-gradient(to bottom, red, yellow);
}
</style>
</head>
<body>

<h1>Linear Gradient - Top to Bottom</h1>
<p>This linear gradient starts red at the top, transitioning to yellow at the bottom:</p>

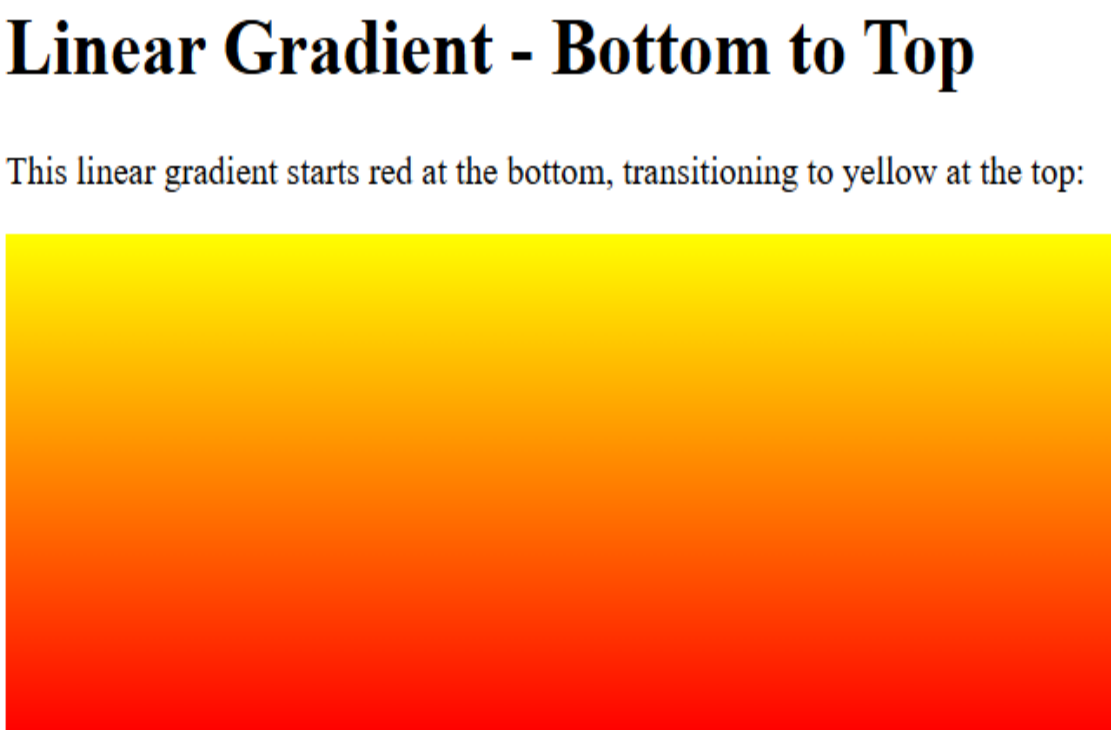
<div id="grad1"></div>

</body>
</html>
```



Direction – Bottom to Top

The following example shows a linear gradient that goes from bottom to top. It starts red, transitioning to yellow:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: linear-gradient(to top, red, yellow);
}
</style>
</head>
<body>

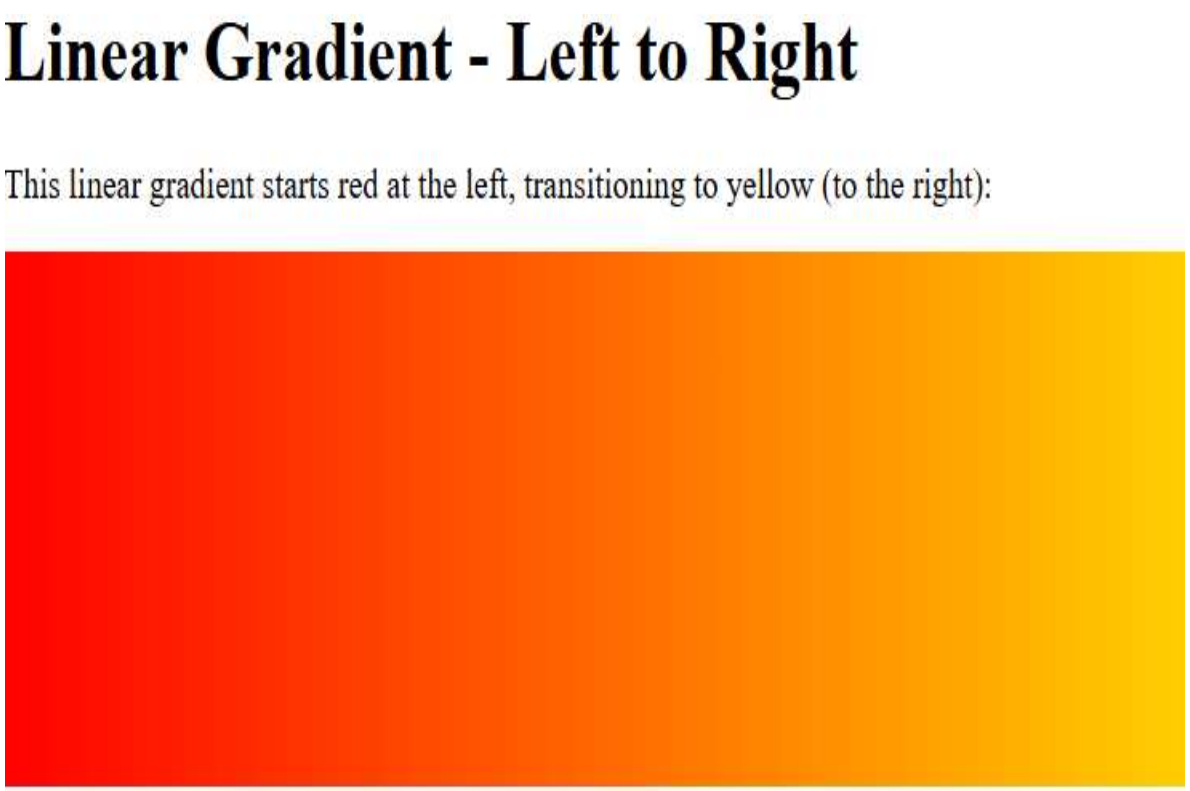
<h1>Linear Gradient - Bottom to Top</h1>
<p>This linear gradient starts red at the bottom, transitioning to yellow at the top:</p>

<div id="grad1"></div>

</body>
</html>
```

Direction – Left to Right

The following example shows a linear gradient that goes from left to right. It starts red, transitioning to yellow:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: linear-gradient(to right, red , yellow);
}
</style>
</head>
<body>

<h1>Linear Gradient - Left to Right</h1>
<p>This linear gradient starts red at the left, transitioning to yellow (to the right):</p>

<div id="grad1"></div>

</body>
</html>
```

Direction – Diagonal

The following example shows a linear gradient that goes from top-left to bottom-right. It starts red, transitioning to yellow:

Linear Gradient - Diagonal

This linear gradient starts red at top left, transitioning to yellow (at bottom right):



```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: linear-gradient(to bottom right, red, yellow);
}
</style>
</head>
<body>

<h1>Linear Gradient - Diagonal</h1>
<p>This linear gradient starts red at top left, transitioning to yellow (at bottom right):</p>

<div id="grad1"></div>

</body>
</html>
```

Linear Gradient - Using Angles

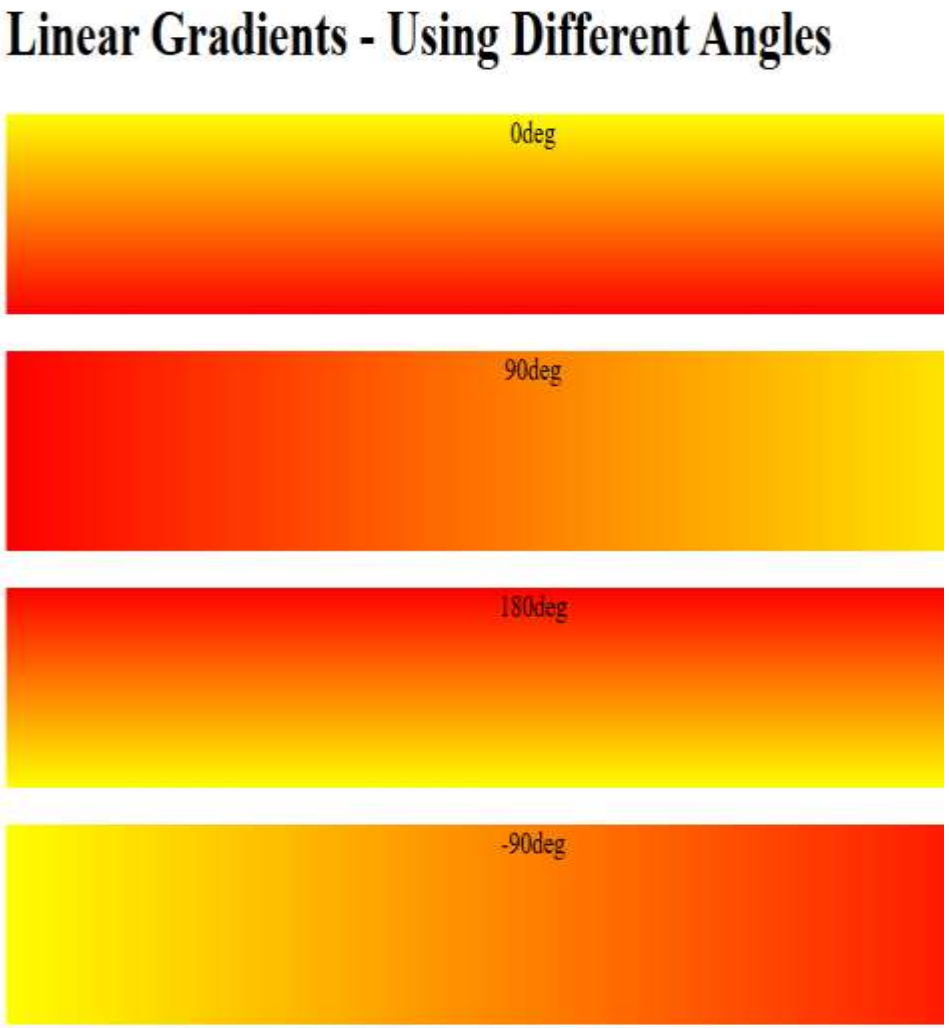
If you want more control over the direction of the gradient, you can define an `angle` parameter, instead of the predefined directions (to bottom, to top, to right, to left, to bottom right, etc.).

- A value of 0deg is equivalent to "to top".
- A value of 90deg is equivalent to "to right".
- A value of 180deg is equivalent to "to bottom".
- A value of 270deg equivalent to "to left"

Syntax

```
background-image: linear-gradient(angle, color-stop1, color-stop2);
```

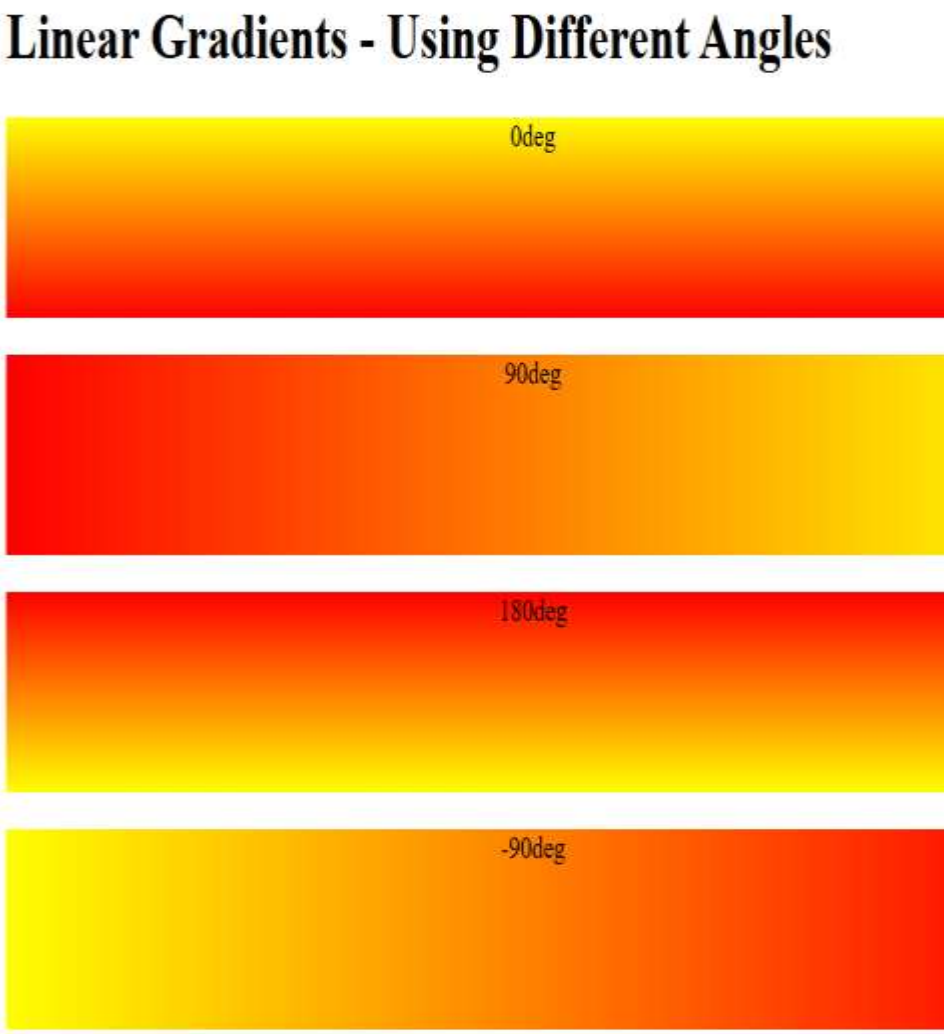
The following example shows how to use angles on linear gradients:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 100px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: linear-gradient(0deg, red, yellow);
}

#grad2 {
  height: 100px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: linear-gradient(90deg, red, yellow);
}

#grad3 {
  height: 100px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: linear-gradient(180deg, red, yellow);
}
```




```
#grad4 {
  height: 100px;
  background-color: red; /* For browsers that do not support
gradients */
  background-image: linear-gradient(-90deg, red, yellow);
}
</style>
</head>
<body>

<h1>Linear Gradients - Using Different Angles</h1>

<div id="grad1" style="text-align:center;">0deg</div><br>
<div id="grad2" style="text-align:center;">90deg</div><br>
<div id="grad3" style="text-align:center;">180deg</div><br>
<div id="grad4" style="text-align:center;">-90deg</div>

</body>
</html>
```

Linear Gradient - Multiple Color Stops

The following example shows a linear gradient (from top to bottom) with multiple color stops:

Linear Gradients - Multiple Color Stops

Note: Color stops are spaced evenly when no percents are specified.

3 Color Stops (evenly spaced):



7 Color Stops (evenly spaced):



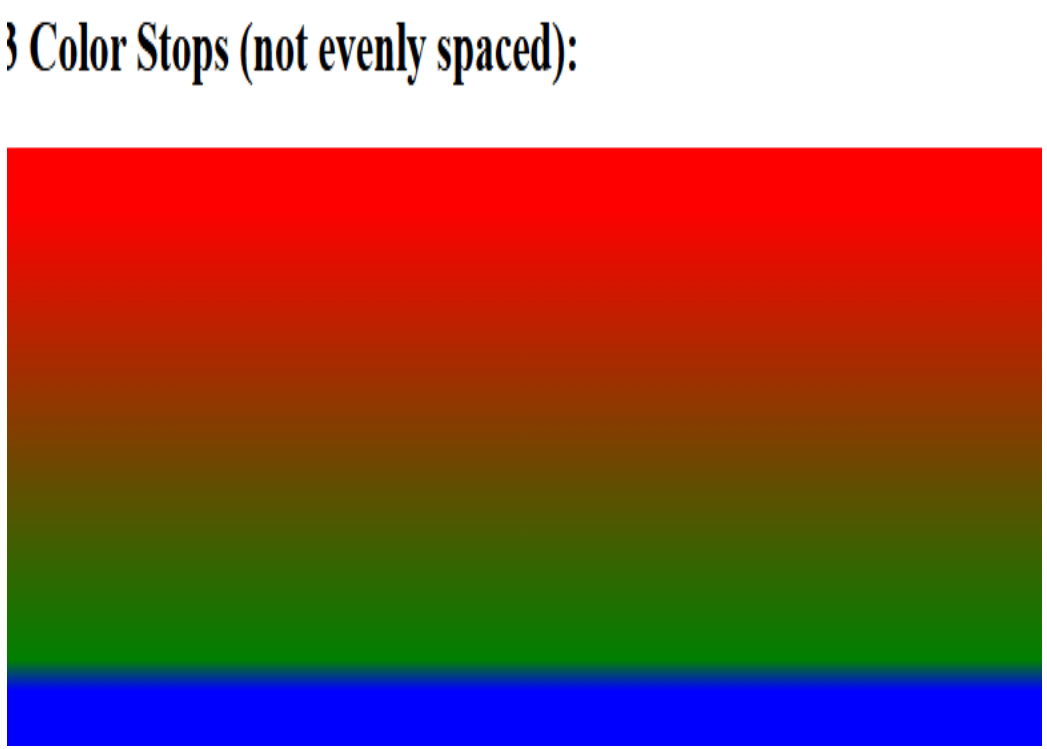
```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients
*/
  background-image: linear-gradient(red, yellow, green);
}

#grad2 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients
*/
  background-image: linear-gradient(red, orange, yellow, green, blue,
indigo, violet);
}

#grad3 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients
*/
  background-image: linear-gradient(red 10%, green 85%, blue 90%);
```

Linear Gradient - Multiple Color Stops

The following example shows a linear gradient (from top to bottom) with multiple color stops:




```
}
</style>
</head>
<body>

<h1>Linear Gradients - Multiple Color Stops</h1>
<p><strong>Note:</strong> Color stops are spaced evenly when no percents are specified.</p>

<h2>3 Color Stops (evenly spaced):</h2>
<div id="grad1"></div>

<h2>7 Color Stops (evenly spaced):</h2>
<div id="grad2"></div>

<h2>3 Color Stops (not evenly spaced):</h2>
<div id="grad3"></div>

</body>
</html>
```

Linear Gradient - Multiple Color Stops

The following example shows a linear gradient (from left to right) with the color of the rainbow and some text:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 100px;
  background-color: red; /* For browsers that do not support gradients
*/
  background-image: linear-gradient(to right, red, orange, yellow,
green, blue, indigo, violet);
}
</style>
</head>
<body>

<div id="grad1" style="text-
align:center;margin:auto;color:#888888;font-size:40px;font-
weight:bold">
Rainbow Background
</div>

</body>
</html>
```

Linear Gradient - Transparency

CSS gradients also support transparency, which can be used to create fading effects.
To add transparency, we use the rgba() function to define the color stops. The last parameter in the rgba() function can be a value from 0 to 1, and it defines the transparency of the color: 0 indicates full transparency, 1 indicates full color (no transparency).

The following example shows a linear gradient that goes from left to right. It starts fully transparent, transitioning to full color red:

```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  background-image: linear-gradient(to right, rgba(255,0,0,0),
rgba(255,0,0,1));
}
</style>
</head>
<body>

<h1>Linear Gradient - Transparency</h1>
<p>To add transparency, we use the rgba() function to define the color stops. The last parameter in the rgba() function can be a value from 0 to 1, and it defines the transparency of the color: 0 indicates full transparency, 1 indicates full color (no transparency).</p>

<div id="grad1"></div>

</body>
</html>
```

Linear Gradient - Transparency

To add transparency, we use the rgba() function to define the color stops. The last parameter in the rgba() function can be a value from 0 to 1, and it defines the transparency of the color: 0 indicates full transparency, 1 indicates full color (no transparency).



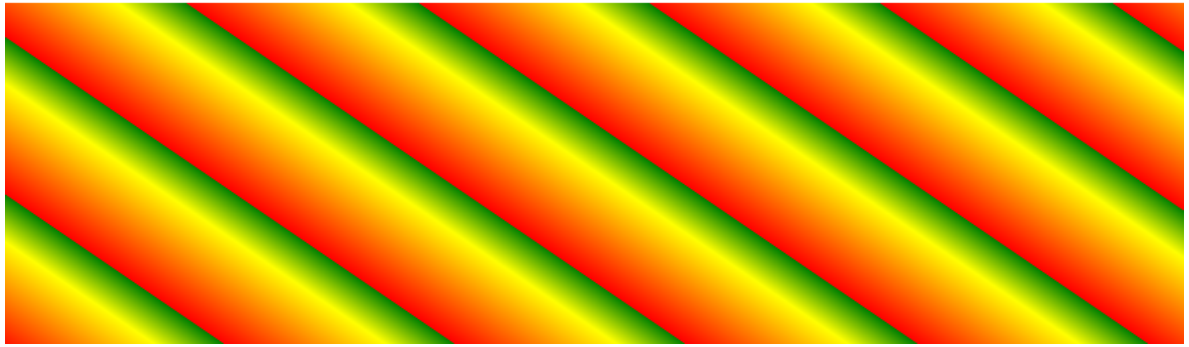
CSS repeating-linear-gradient() Function

CSS `repeating-linear-gradient()` function is used to repeat linear gradients:

Repeating Linear Gradient



A repeating gradient on 45deg axe starting red and finishing green:



```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: repeating-linear-gradient(red, yellow 10%, green 20%);
}

#grad2 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: repeating-linear-gradient(45deg,red,yellow 7%,green 10%);
}

#grad3 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
```

```
background-image: repeating-linear-gradient(90deg,red,yellow 7%,green 10%);
}

#grad5 {
  height: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: repeating-linear-gradient(45deg, red 0px, red 10px, yellow 10px, yellow 20px);
}
</style>
</head>
<body>

<h1>Repeating Linear Gradient</h1>

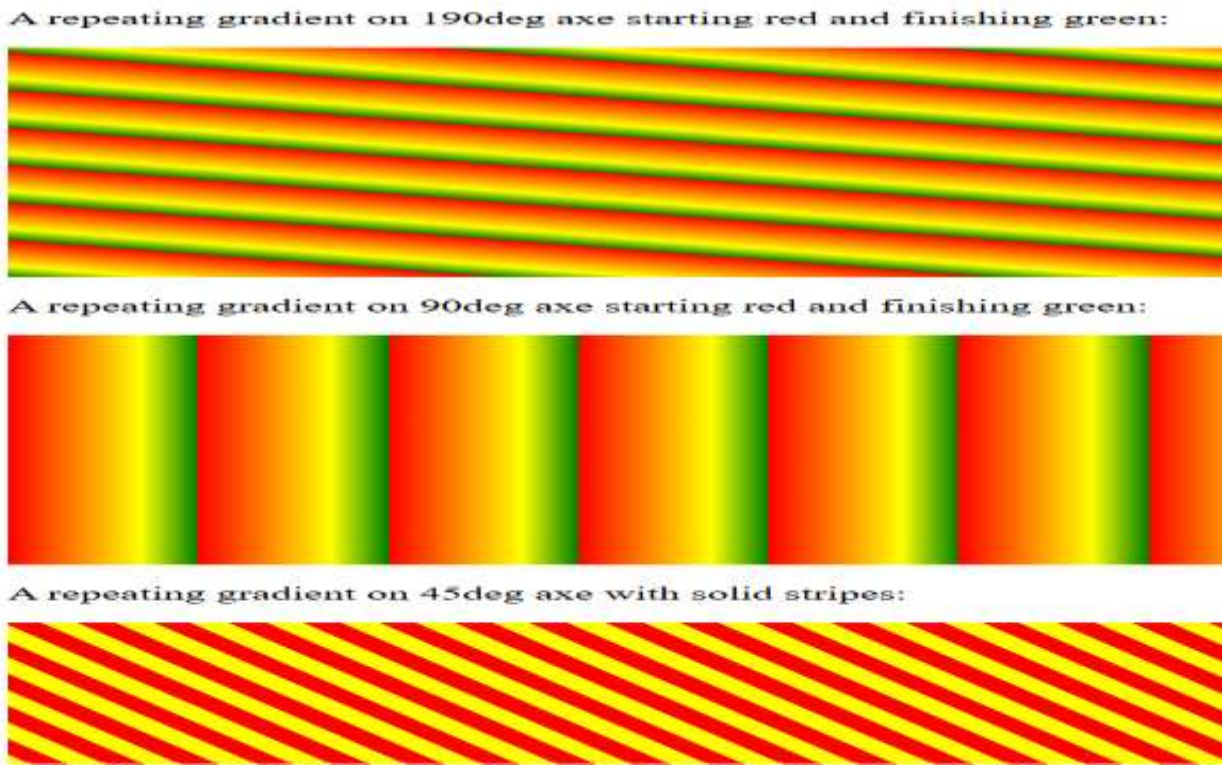
<div id="grad1"></div> <p>A repeating gradient on 45deg axe starting red and finishing green:</p>
<div id="grad2"></div>

<p>A repeating gradient on 190deg axe starting red and finishing green:</p>
<div id="grad3"></div>

<p>A repeating gradient on 90deg axe starting red and finishing green:</p>
<div id="grad4"></div>

<p>A repeating gradient on 45deg axe with solid stripes:</p>
<div id="grad5"></div>

</body>
</html>
```



CSS radial-gradient() Function

The CSS `radial-gradient()` function creates a radial gradient.
A radial gradient defines a color transition that goes goes out from a central point.
A radial gradient requires at least two color stops. Color stops are the colors you want to render smooth transitions among.

Syntax

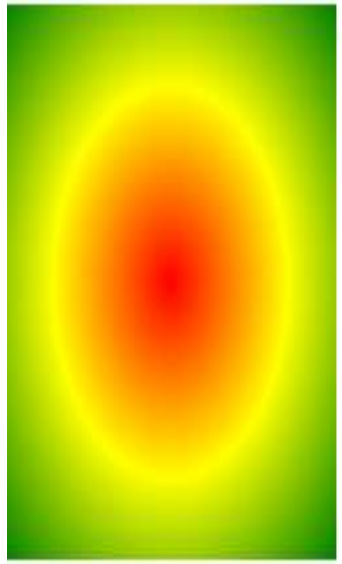
```
background-image: radial-gradient(shape size at position, start-color, ..., last-color);
```

By default, `shape` is ellipse, `size` is farthest-corner, and `position` is center.

Radial Gradient – Evenly Spaced Color Stops (this is default)

The following example shows a radial gradient with evenly spaced color stops:

Radial Gradient - Evenly Spaced Color Stops



```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 150px;
  width: 200px;
  background-color: red; /* For browsers that do not support
gradients */
  background-image: radial-gradient(red, yellow, green);
}
</style>
</head>
<body>

<h1>Radial Gradient - Evenly Spaced Color Stops</h1>

<div id="grad1"></div>

</body>
</html>
```

Radial Gradient – Differently Spaced Color Stops

The following example shows a radial gradient with differently spaced color stops:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 150px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: radial-gradient(red 5%, yellow 15%, green 60%);
}
</style>
</head>
<body>

<h1>Radial Gradient - Differently Spaced Color Stops</h1>

<div id="grad1"></div>

</body>
</html>
```

Radial Gradient – Set Shape

The `shape` parameter defines the shape of the gradient. It can take one of the following values:

- **ellipse** (this is default)
- **circle**

The following example shows a radial gradient with the shape of a circle:

```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 150px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: radial-gradient(red, yellow, green);
}

#grad2 {
  height: 150px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: radial-gradient(circle, red, yellow, green);
}
</style>
</head>
<body>

<h1>Radial Gradient - Shapes</h1>

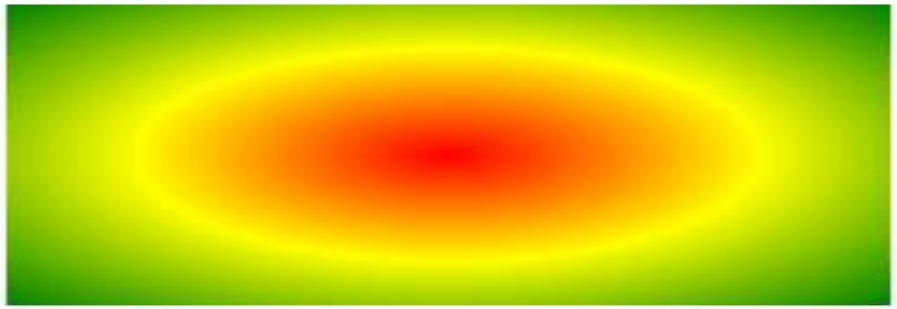
<h2>Ellipse (this is default):</h2>
<div id="grad1"></div>

<h2><strong>Circle:</strong></h2>
<div id="grad2"></div>

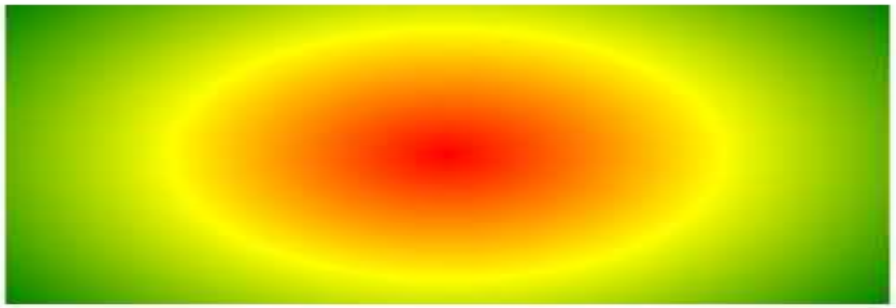
</body>
</html>
```

Radial Gradient - Shapes

Ellipse (this is default):



Circle:



Radial Gradient - The size Parameter

The `size` parameter defines the size of the gradient'ending shape.
It can take one of the following values:

- **closest-side**
- **farthest-side**
- **closest-corner**
- **farthest-corner** (this is default)

Example
A radial gradient with different size keywords:

```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 150px;
  width: 350px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: radial-gradient(closest-side at 70% 60%, red, yellow, black);
}

#grad2 {
  height: 150px;
  width: 350px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: radial-gradient(farthest-side at 70% 60%, red, yellow, black);
}
```

```
#grad3 {
  height: 150px;
  width: 350px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: radial-gradient(closest-corner at 70% 60%, red, yellow, black);
}

#grad4 {
  height: 150px;
  width: 350px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: radial-gradient(farthest-corner at 70% 60%, red, yellow, black);
}
</style>
</head>
<body>

<h1>Radial Gradients - Different size keywords</h1>

<h2>closest-side:</h2>
<div id="grad1"></div>

<h2>farthest-side:</h2>
<div id="grad2"></div>

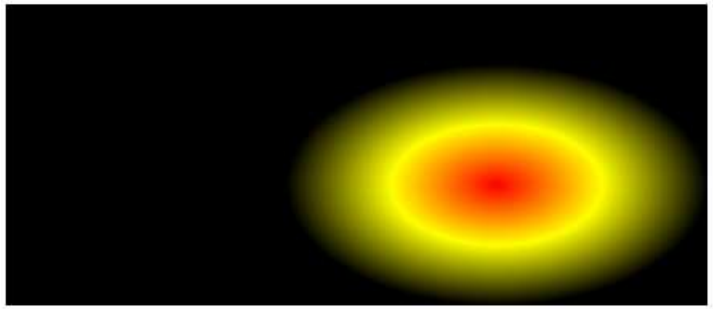
<h2>closest-corner:</h2>
<div id="grad3"></div>

<h2>farthest-corner (default):</h2>
<div id="grad4"></div>

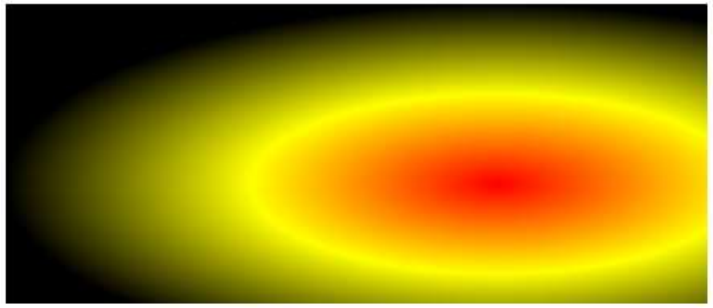
</body>
</html>
```

Radial Gradients - Different size keywords

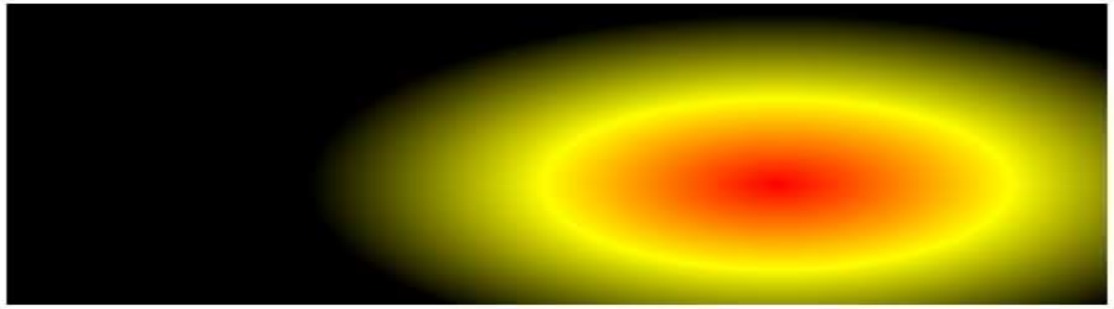
closest-side:



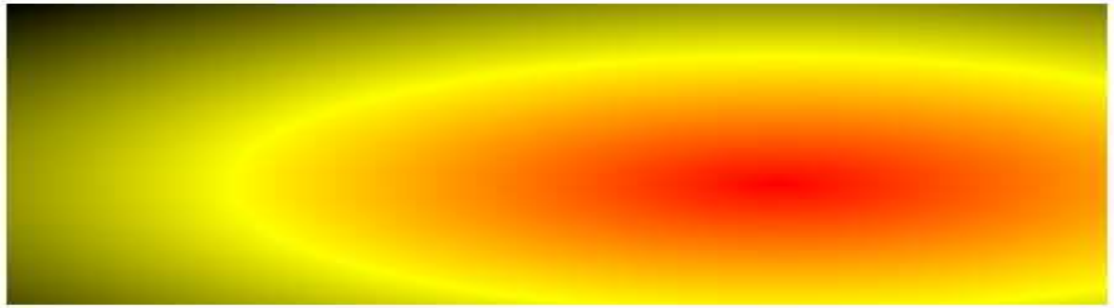
farthest-side:



closest-corner:

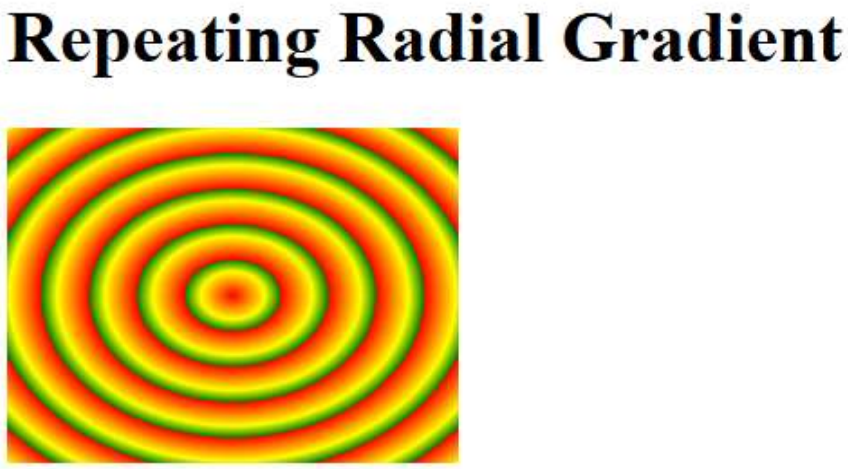


farthest-corner (default):



CSS repeating-radial-gradient() Function

The CSS `repeating-radial-gradient()` function is used to repeat radial gradients:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 150px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: repeating-radial-gradient(red, yellow 10%, green 15%);
}
</style>
</head>
<body>

<h1>Repeating Radial Gradient</h1>

<div id="grad1"></div>

</body>
</html>
```

CSS conic-gradient() Function

The CSS `conic-gradient()` function creates a conic gradient.
A conic gradient is a gradient with color transitions rotated around a center point.
To create a conic gradient you must define at least two colors.

Syntax

```
background-image: conic-gradient([from angle]
[at position,] color [degree], color [degree], ...);
```

By default, `angle` is 0deg and `position` is center.
If no `degree` is specified, the colors will be spread equally around the center point.

Conic Gradient: Three Colors

The following example shows a conic gradient with three colors:

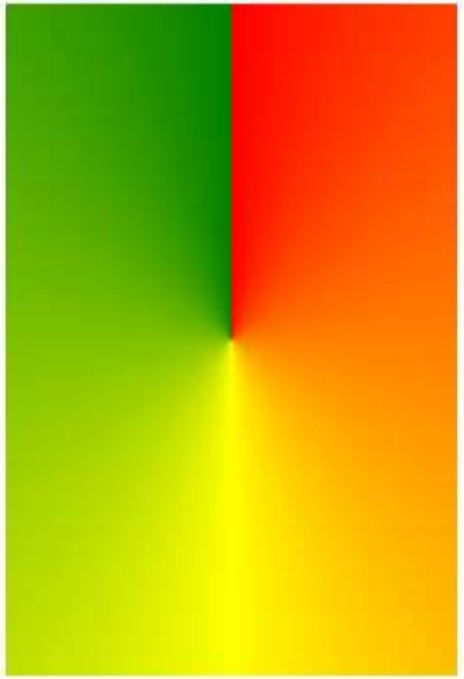
```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support
gradients */
  background-image: conic-gradient(red, yellow, green);
}
</style>
</head>
<body>

<h1>Conic Gradient - Three Colors</h1>

<div id="grad1"></div>

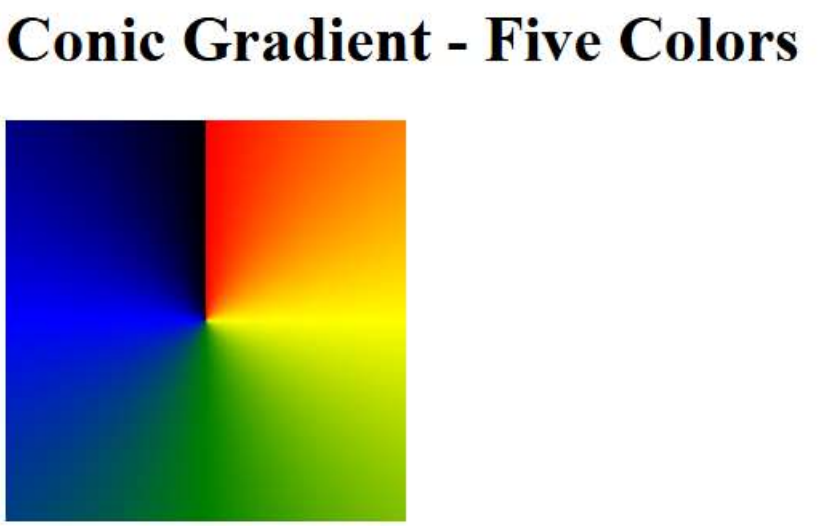
</body>
</html>
```

Conic Gradient - Three Colors



Conic Gradient: Five Colors

The following example shows a conic gradient with five colors:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(red, yellow, green, blue, black);
}
</style>
</head>
<body>

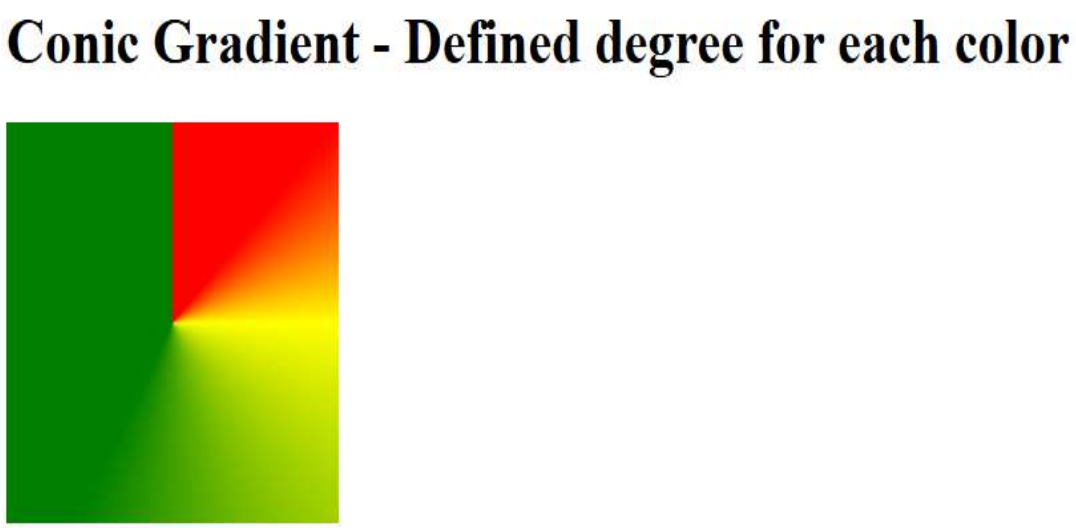
<h1>Conic Gradient - Five Colors</h1>

<div id="grad1"></div>

</body>
</html>
```

Conic Gradient: Three Colors and Degrees

The following example shows a conic gradient with three colors and a degree for each color:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(red 45deg, yellow 90deg, green 210deg);
}
</style>
</head>
<body>

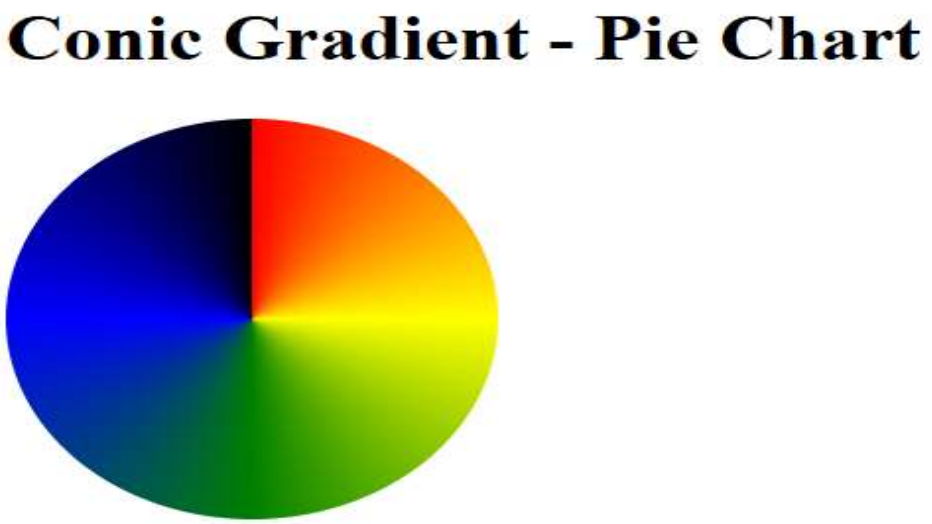
<h1>Conic Gradient - Defined degree for each color</h1>

<div id="grad1"></div>

</body>
</html>
```

Create Pie Charts

Just add border-radius: 50% to make the conic gradient look like a pie:




```
!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(red, yellow, green, blue, black);
  border-radius: 50%;
}
</style>
</head>
<body>

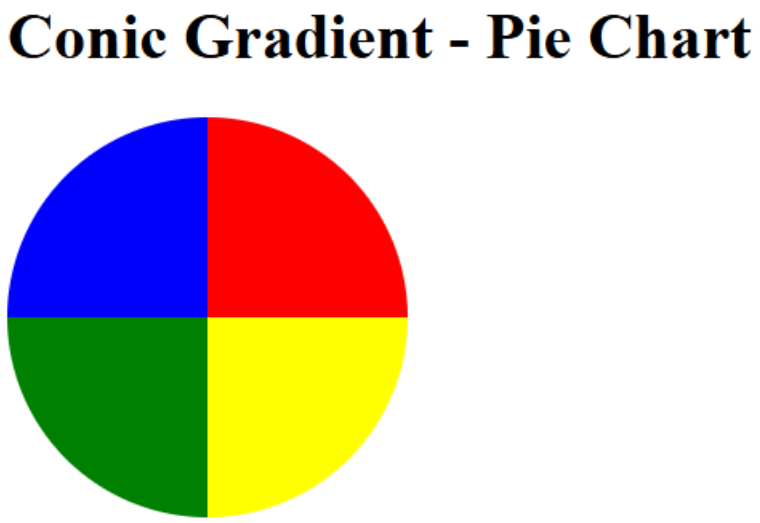
<h1>Conic Gradient - Pie Chart</h1>

<div id="grad1"></div>

</body>
</html>
```

Create Pie Charts

Here is another pie chart with defined degrees for all the colors:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(red 0deg, red 90deg, yellow 90deg, yellow 180deg, green 180deg, green 270deg, blue 270deg);
  border-radius: 50%;
}
</style>
</head>
<body>

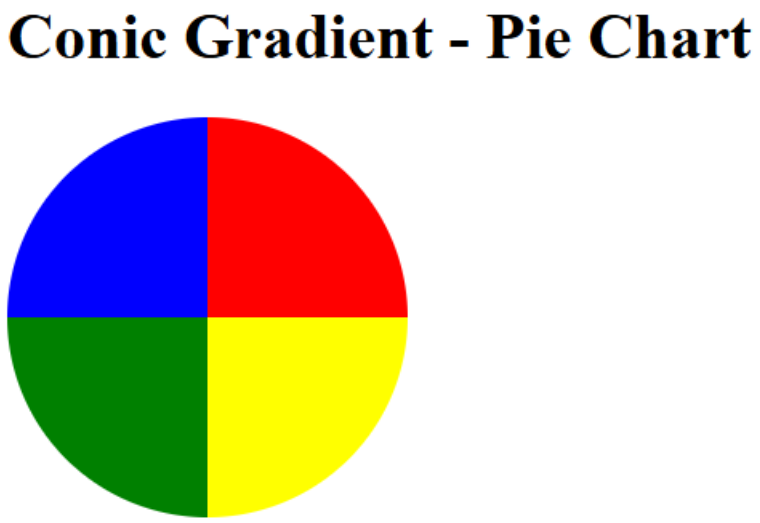
<h1>Conic Gradient - Pie Chart</h1>

<div id="grad1"></div>

</body>
</html>
```

Create Pie Charts

Here is another pie chart with defined degrees for all the colors:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(red 0deg, red 90deg, yellow 90deg, yellow 180deg, green 180deg, green 270deg, blue 270deg);
  border-radius: 50%;
}
</style>
</head>
<body>

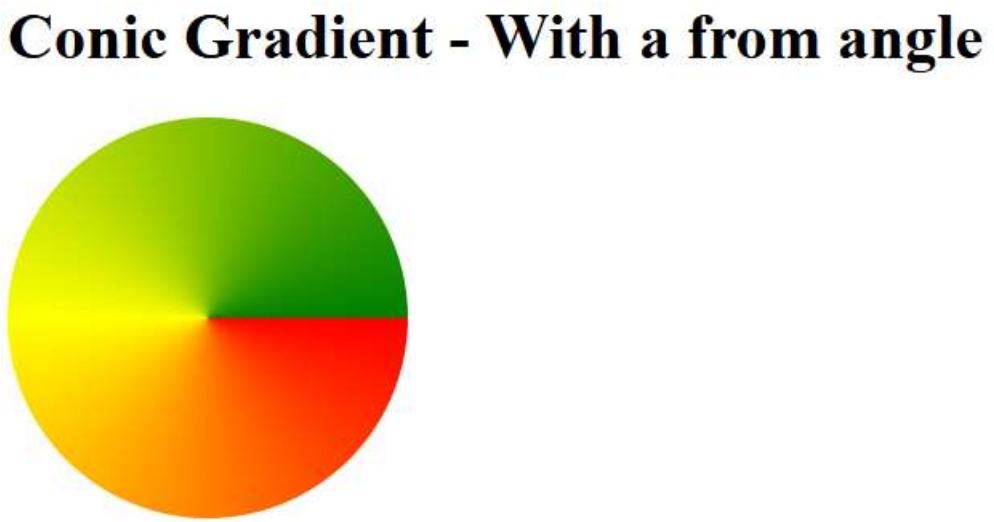
<h1>Conic Gradient - Pie Chart</h1>

<div id="grad1"></div>

</body>
</html>
```

Conic Gradient With Specified From Angle

The [from *angle*] specifies an angle that the entire conic gradient is rotated by.
The following example shows a conic gradient with a from angle of 90deg




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(from 90deg, red, yellow, green);
  border-radius: 50%;
}
</style>
</head>
<body>

<h1>Conic Gradient - With a from angle</h1>

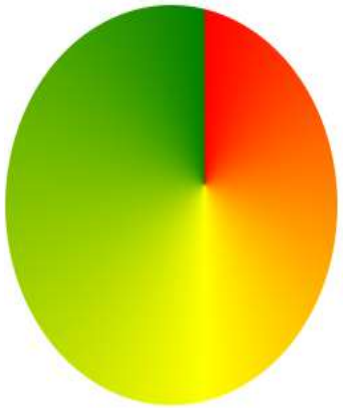
<div id="grad1"></div>

</body>
</html>
```

Conic Gradient With Specified Center Position

The [at *position*] specifies the center of the conic gradient.
The following example shows a conic gradient with a center position of 60% 45%:

Conic Gradient - With a specified center position



```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: conic-gradient(at 60% 45%, red, yellow, green);
  border-radius: 50%;
}
</style>
</head>
<body>

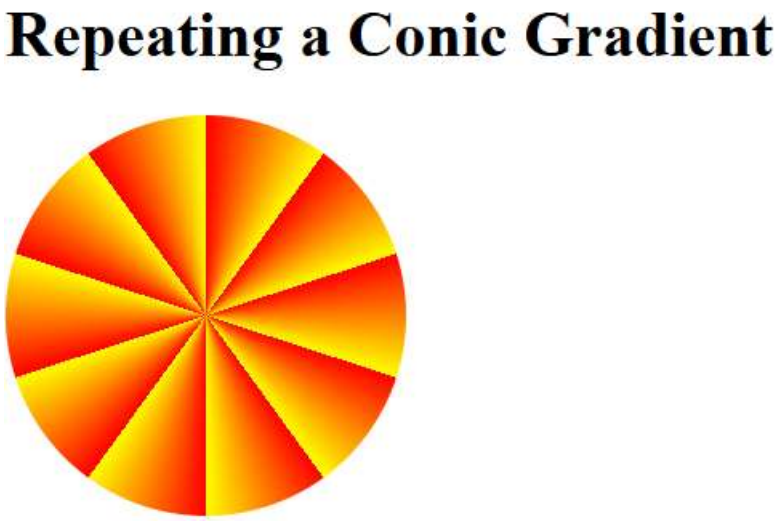
<h1>Conic Gradient - With a specified center position</h1>

<div id="grad1"></div>

</body>
</html>
```

CSS repeating-conic-gradient() Function

The CSS repeating-conic-gradient() function is used to repeat conic gradients:




```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: repeating-conic-gradient(red 10%, yellow 20%);
  border-radius: 50%;
}
</style>
</head>
<body>

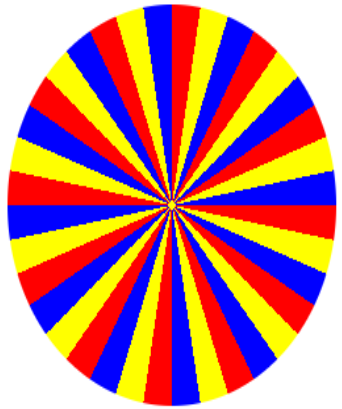
<h1>Repeating a Conic Gradient</h1>

<div id="grad1"></div>

</body>
</html>
```

Here is a repeating conic gradient with defined color-starts and color-stops:

Repeating a Conic Gradient with color-starts and -stops



```
<!DOCTYPE html>
<html>
<head>
<style>
#grad1 {
  height: 200px;
  width: 200px;
  background-color: red; /* For browsers that do not support gradients */
  background-image: repeating-conic-gradient(red 0deg 10deg, yellow 10deg 20deg, blue 20deg 30deg);
  border-radius: 50%;
}
</style>
</head>
<body>

<h1>Repeating a Conic Gradient with color-starts and -stops</h1>

<div id="grad1"></div>

</body>
</html>
```

CSS Properties

- **CSS Fonts:**
  - **color:** The CSS color property **defines the text color** to be used.

```
h1 { color: red; }     //specify color name
h2 { color: #9000A1; }   //specify by hexadecimal value
p { color:rgb(0, 220, 98); }   //by RGB color combinations
```

  - **font-family:** The CSS font-family property defines the **font to be used.**
  - **font-size** property defines the **text size** to be used.

```
font-size: 30px; or  font-size: 200%;
```

  - **font-style:** The CSS font-style property defines the **style of font to be used**.

```
font-style: italic
```

- **border**: defines a border around an HTML element.
  - border-style {border-style: none;} {border-style: dotted;}
  - border-color border-color: red;
  - border-width border-width: 1px;
- **padding**: defines a padding (space) between the text and the border.

{ padding-top: 50px; padding-right: 100px; padding-bottom: 150px; }

- **margin:** defines a margin (space) outside the border.

{ margin-top: 50px; margin-bottom: 50px; margin-right: 100px; }

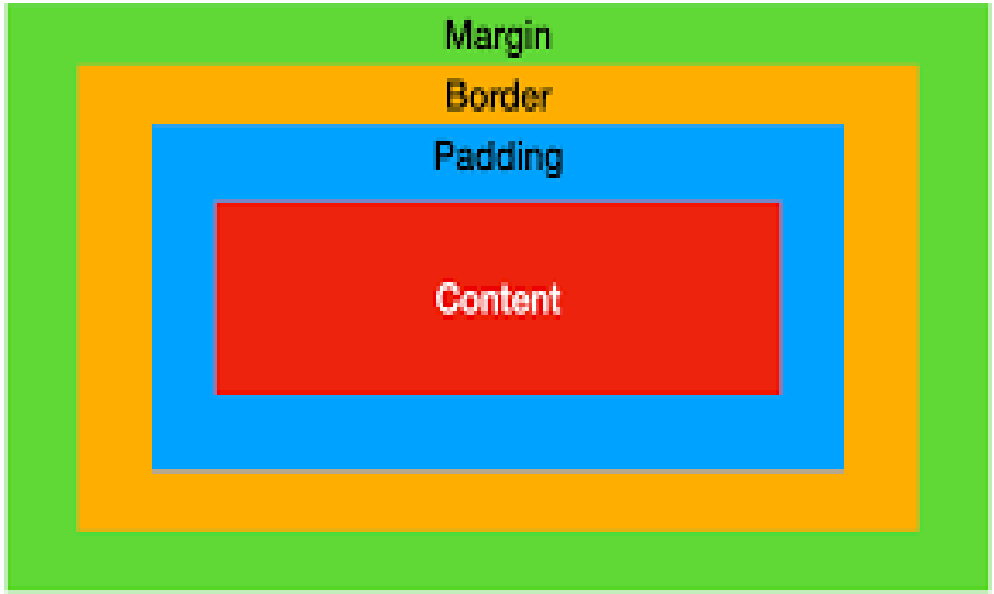
```
h1
   { color: blue;
    font-family: verdana;
  }
p {
 color: red;
  font-family: courier;
  font-size: 160%;
 }
```

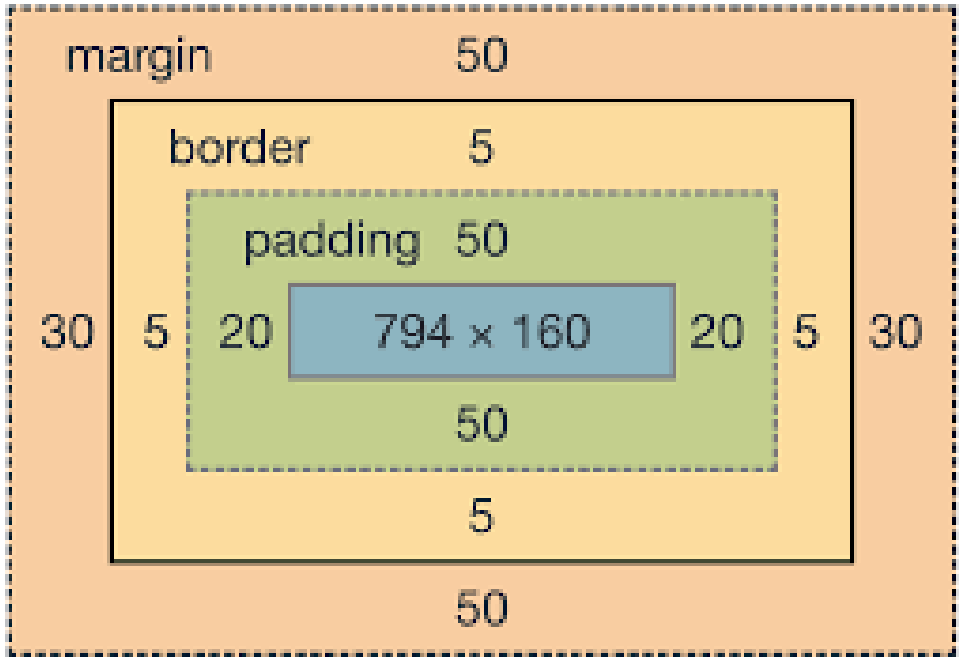
```
h1{ color: blue;
    font-family:'Courier New';
 }
p { color: red;
  font-size: 200%;
  border: 2px solid powderblue;
  padding: 30px;
  margin: 50px; }
```

CSS Box Model

- In CSS, the term "**box model**" is used when talking about **design and layout**.

- The CSS box model is **essentially a box** that wraps around every HTML element.
- It consists of: **margins, borders, padding, and the actual content**. The image below illustrates the box model:





- **Content** - The **content of the box**, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - **Clears an area outside the border**. The margin is transparent

Demonstration of the box model

```
<!- box model -->
<html>
<head>
<style>
div
{
  background-color: lightgrey;
  width: 300px;
  border: 15px solid green;
  padding: 50px;
  margin: 20px;
}
</style>
</head>
<body>
```

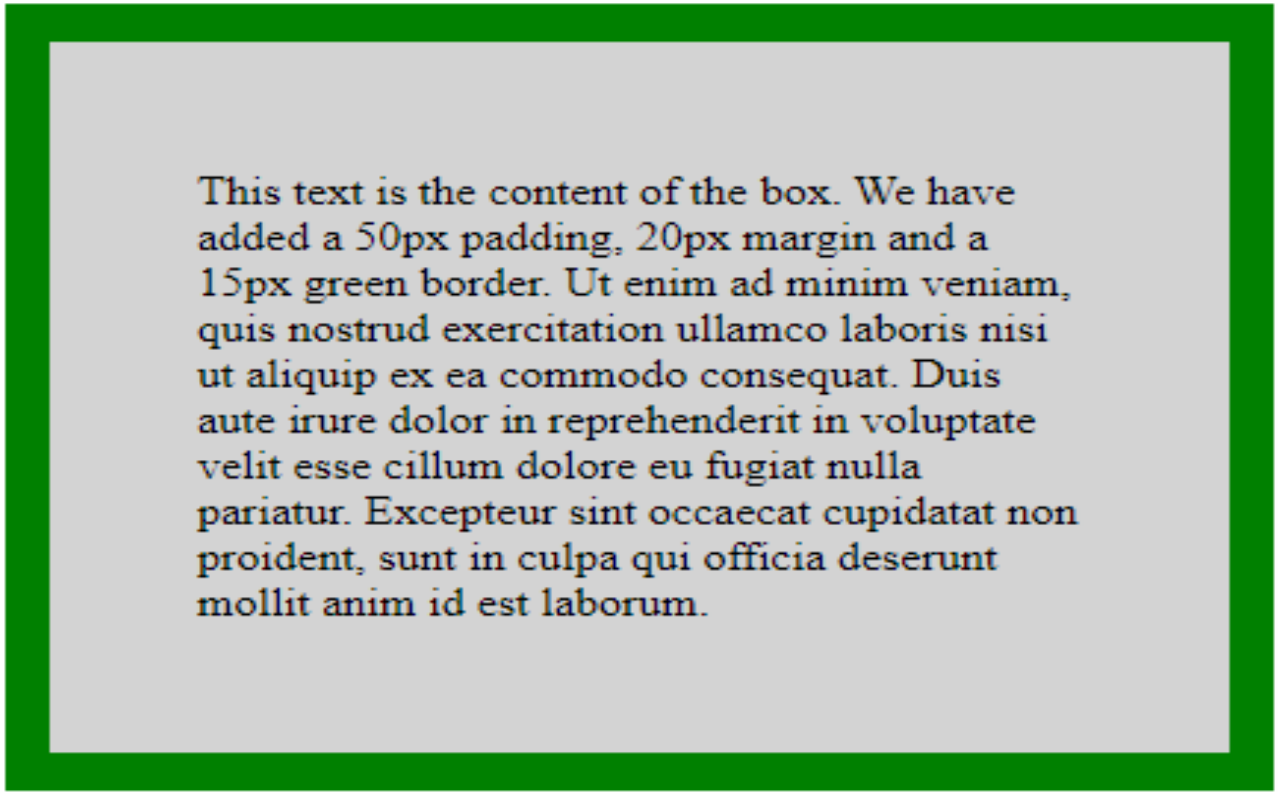
```
<h2>Demonstrating the Box Model</h2>

<div>
This text is the content of the box. We have added a 50px
padding, 20px margin and a 15px green border.
This text is the content of the box. We have added a 50px
padding, 20px margin and a 15px green border.

This text is the content of the box. We have added a 50px
padding, 20px margin and a 15px green border.

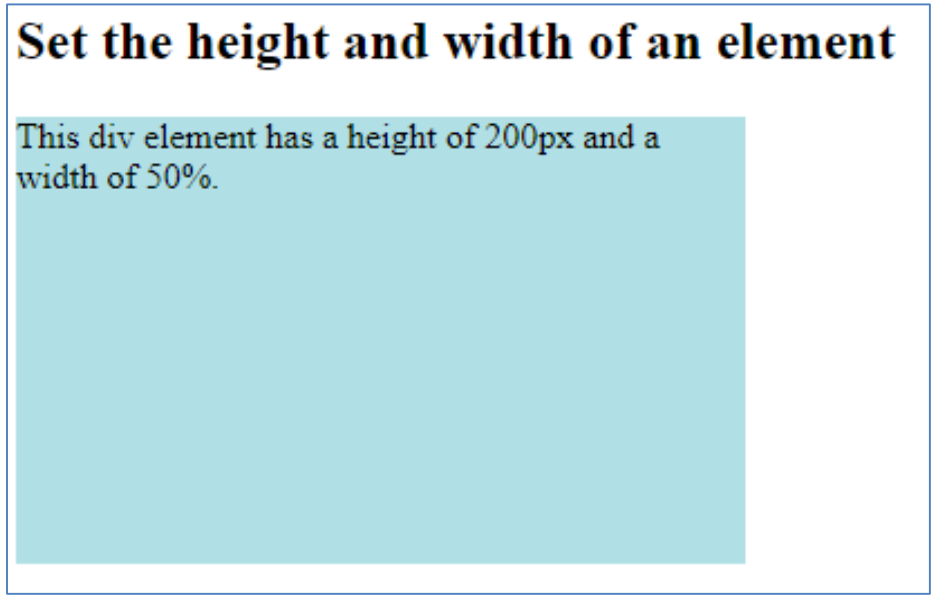
</div>

</body>
</html>
```


This text is the content of the box. We have added a 50px padding, 20px margin and a 15px green border. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

CSS Height, Width and Max-width

- The CSS **height and width** properties are used to set the **height and width of an element.**
- The CSS max-width property is used to **set the maximum width** of an element.
- This element has a height of 50 pixels and a width of 100%.




```
div {
  height: 200px;
  width: 50%;
  background-color: aqua;
  max-width: fit-content;
}
<body>
<h2>Set the height and width of an element</h2>
<div>This div element has a height of 200px and a width of
50%.</div>
</body>
```

CSS Box Sizing

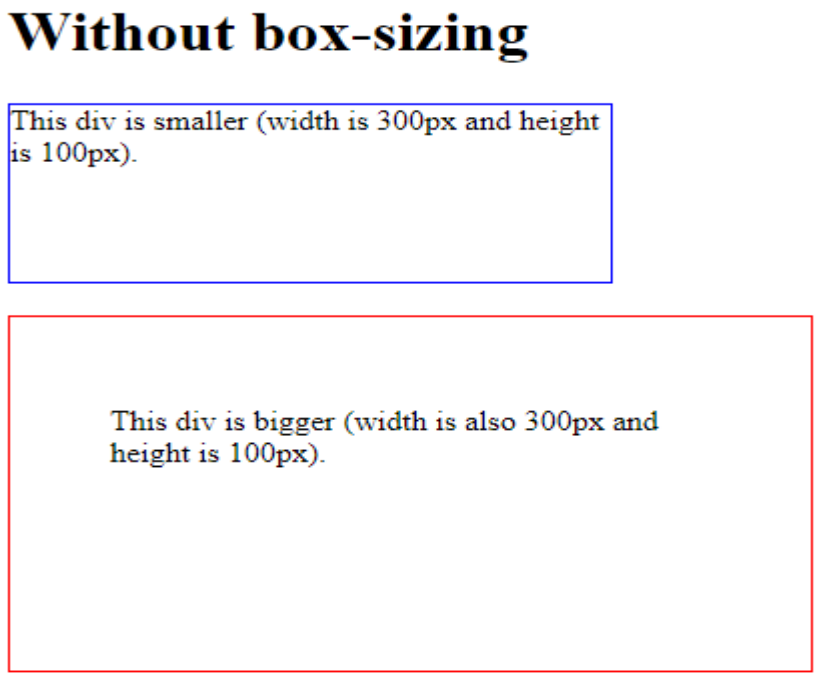
- The CSS **box-sizing property** allows us to include the **padding and border** in an element's **total width and height.**

Without the CSS box-sizing Property:

- By default, the width and height of an element is calculated like this:
  - **width + padding + border** = **actual width** of an element
  - **height + padding + border** = **actual height** of an element

```
.div1 {
  width: 300px;
  height: 100px;
  border: 1px solid blue;
}

.div2 {
  width: 300px;
  height: 100px;
  padding: 50px;
  border: 1px solid red;
}
```



Border Radius

- The CSS box model includes the **border radius** property, which allows you to **specify the curvature of the corners of an element's border.**

- Specify the value for this property in several ways, including:
  - A **single value** □ **sets the same radius for all four corners of the border.**
  - **Two values** □ set different radii for the **top and bottom** corners and the **left and right** corners.
  - **Four values** □ set **different radii for each individual corner** of the border.

SINGLE VALUE
For example, to set a border radius of **10 pixels for all corners** of an element's border, you would use the following CSS code:

```
border-radius: 10px;
```

TWO VALUES
To set different radii for the **top and bottom corners**, and the **left and right corners**, you would use **two values** like this:

```
border-radius: 10px 20px;
```

FOUR VALUES
This would set a radius of 10 pixels for the **top and bottom** corners, and a radius of 20 pixels for the **left and right corners.**

```
border-radius: 10px 20px 30px 40px;
```

```
<!- box model with border radius-->
<html>
<head>
    <title>Border Radius Example</title>
    <style>
        .box {
            background-color: lightblue;
            padding: 20px;
            border-radius: 10px;
        }
    </style>
</head>
<body>
    <div class="box">
        <p>This is an example of an element with
rounded corners.</p>
    </div>
</body>
</html>
```

This is an example of an element with rounded corners.

```
.box1 {
background-color: lightblue;
padding: 20px;
border-radius: 10px 30px;
/*top=10 bottom=30 left=10 right=30*/
}
```

top=10 bottom=30 left=10 right=30

```
.box3 {
background-color: lightblue;
padding: 20px;
border-radius: 10px 20px 40px 30px;
}
```

top-left=10 top-right=20 bottom-right=40 bottom-left=30

Padding

- In CSS, the **padding** property is used to **add space between an element's content and its border**.
- The padding property can be **applied to any HTML element, including text, images, and containers.**

The padding property can be set in several ways:

- **padding: value;** sets the **same padding value for all four sides** of the element.

Ex.
padding: 10px;
 add **10** pixels of padding to the **top, bottom, left, and right** sides of the element.

- **padding: vertical horizontal;** sets the **padding value for the top and bottom, and left and right sides** of the element separately.

Ex.
padding: 10px 20px
 add 10 pixels of padding to the **top and bottom**,
20 pixels of padding to the **left and right**.

- **padding: top right bottom left;** sets the padding value for each side of the element separately.

Ex.
padding: 10px 20px 30px 40px
 add 10 pixels of padding to the **top**,
20 pixels of padding to the **right**,
30 pixels of padding to the **bottom**
40 pixels of padding to the **left**

```
<!-- box model with padding-->
<html>
<head>
<style>
.box3
{
    background-color: lightblue;
    padding: 20px;
    border-radius: 10px;
}
</style>
</head>
<body>
<div class="box3">
 padding 20px for all sides
  i.e. top, bottom, left and right
</div>
</body>
</html>
```

padding 20px for all sides i.e. top, bottom, left and right

```
.box3
{
    background-color: lightblue;
    padding: 10px 20px;
    border-radius: 10px;
}
```

padding 10 pixels for top and botton
padding 20 pixels for left and right

```
.box3
{   background-color: lightblue;
    padding: 10px 20px 30px 40px;
    border-radius: 10px;
}
```

10 pixels of padding to the top
20 pixels of padding to the right
30 pixels of padding to the bottom
40 pixels of padding to the left

Margins

- In CSS, the **margin** property is used to **add space between an element's border and the adjacent elements**
- The margin property can be applied to any HTML element, including text, images, and containers.

The **margin** property can be set in several ways:

- **margin: value; sets the same margin value for all four sides of the element**

For example,
margin: 10px;
 add 10 pixels of margin to the top, bottom, left, and right sides of the element.

- **margin: vertical horizontal; sets the margin value for the top and bottom, and left and right sides of the element separately..**

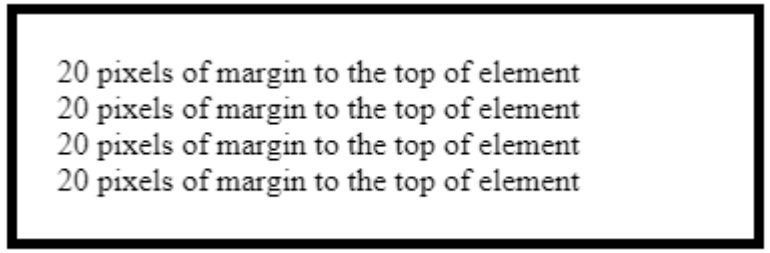
For example,
margin: 10px 20px;
 add 10 pixels to the top and bottom,
20 pixels to the left and right.

- **margin: top right bottom left; sets the margin value for each side of the element separately.**

For example,
margin: 10px 20px 30px 40px;
 add 10 pixels of margin to the top,
 20 pixels of margin to the right,
 30 pixels of margin to the bottom, and
 40 pixels of margin to the left.

```
<!-- box model with margin->
<html>
<head>
<style>
.box {
border: 5px solid black;
padding: 20px;
margin: 20px;
}
</style>
</head>
<body>
<div class="box">
     20 pixels of margin to the top, bottom, left
and righ of element<br>

</div>
</body>
</html>
```



Auto Margin

- The **auto value** for the margin property can be used to horizontally center an element within its parent container.
- Here's an example of how to use **margin: auto** to center an element horizontally:
- In this example, we have defined a element with a class of "box".

margin: auto;

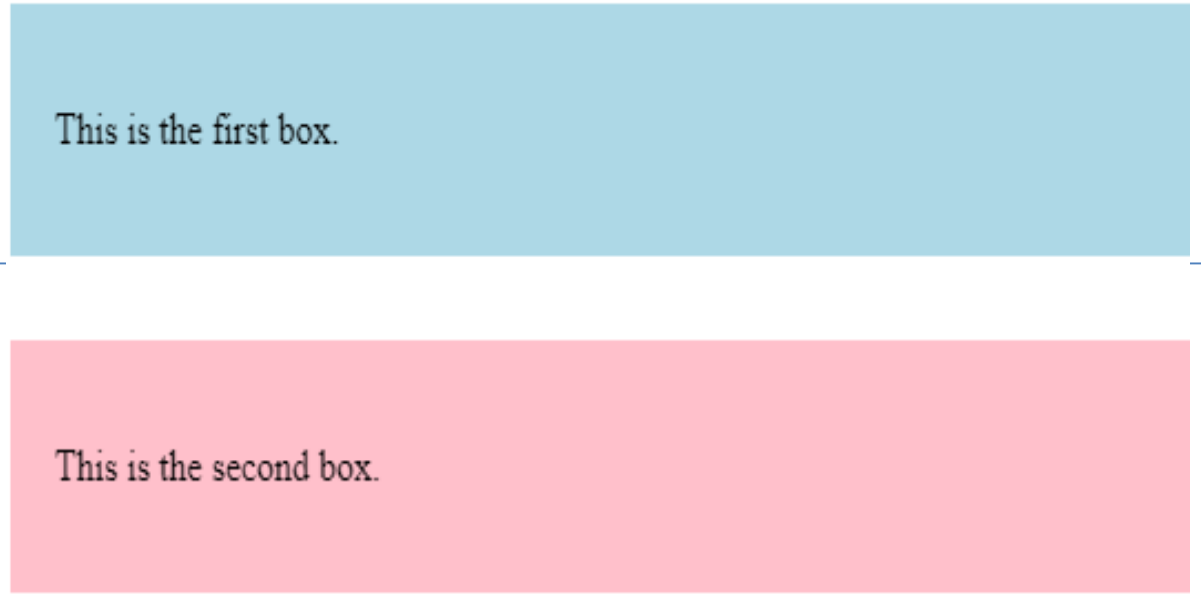
```
 .box {
background-color: lightblue;
padding: 20px;
margin: auto;
width: 50%;
}
```

```
<!– auto margin demo->
<html>
<head>
        <title>Auto Margin Example</title>
        <style>
        </style>
</head>
<body>
        <div class="box">
                <p>This is an example of an
element centered using auto margins.</p>
        </div>
</body>
</html>
```

Margin collapse

- In CSS, margin collapse is a behavior where **adjacent margins between two elements collapse into a single margin.**
- This can affect the layout of elements in unexpected ways
- Ex. Here two ** **elements with different background colors, padding, and margins.

- The first has a margin-bottom of 20 pixels,
- second element has a margin-top of 30 pixels.




```
<html>
<head> <style>
        .box1 {
                background-color: lightblue;
                margin-bottom: 20px;
                padding: 20px;
                }
        .box2 {
                background-color: pink;
                margin-top: 30px;
                padding: 20px;
                }
        </style>
</head>
<body>
<div class="box1">
        <p>This is the first box.</p>
</div>
<div class="box2">
        <p>This is the second box.</p>
        </div> </body>
</html>
```

This is the first box.

This is the second box.

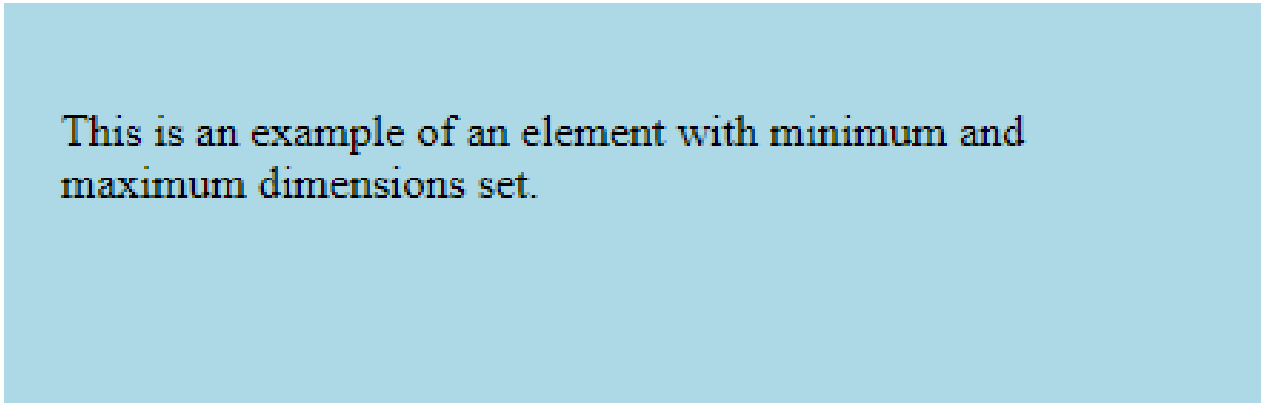
- Here the margin between the two boxes is **not the sum (20px + 30px = 50px),** but instead is **the max value** of the two margins (30px).
- This is because the **margins have collapsed** into a single margin between the two boxes.

- Margin collapse can occur in other situations as well, such as between siblings, or between a parent and its first or last child element.
- To prevent margin collapse, you can use techniques such as **setting a padding value on the parent element**, or **using a border or outline property.**

```
<html>
<head> <style>
        .box1 {
                background-color: lightblue;
                margin-bottom: 20px;
                padding: 20px;
                }
        .box2 {
                background-color: pink;
                margin-top: 30px;
                padding: 20px;
                }
        </style>
</head>
<body>
<div class="box1">
        <p>This is the first box.</p>
</div>
<div class="box2">
        <p>This is the second box.</p>
        </div> </body>
</html>
```

Minimum and Maximum Height and Width

- In CSS, the min-height, max-height, min-width, and max-width properties can be used to set minimum and maximum dimensions for an element in the box model.
- We have set the **min-height** property to 100px and the **max-height** property to 300px.
- We have also set the **min-width** property to 200px and the **max-width** property to 400px.

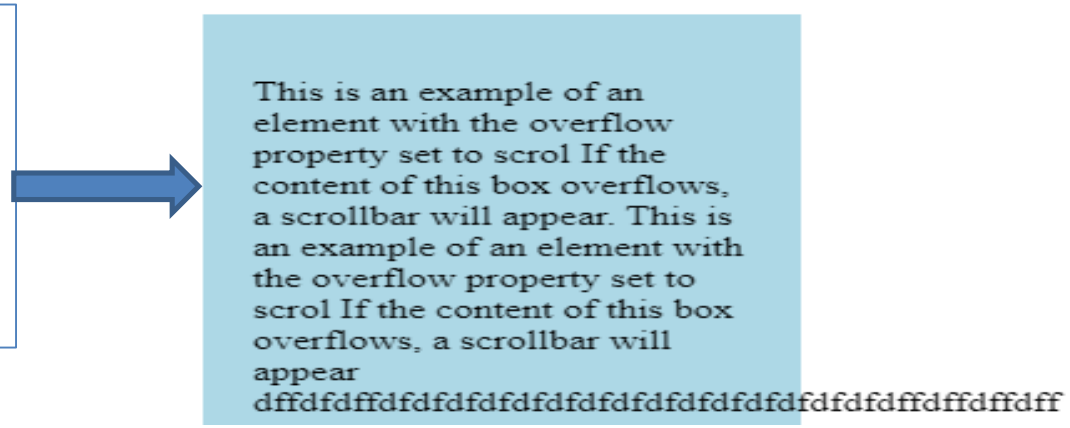



```
<html>
<head>
    <title>Minimum and Maximum Dimensions
Example</title>
    <style>
    .box {
        background-color: lightblue;
        padding: 20px;
        min-height: 100px;
        max-height: 300px;
        min-width: 200px;
        max-width: 400px;
        }
    </style>
</head>
<body>
<div class="box">
    <p>This is an example of an element with
minimum and maximum dimensions set.</p>
</div> </body>
</html>
```

Overflow

- **visible:** This is the default value. Content overflows outside the box model and can overlap other elements.

  overflow:visible:

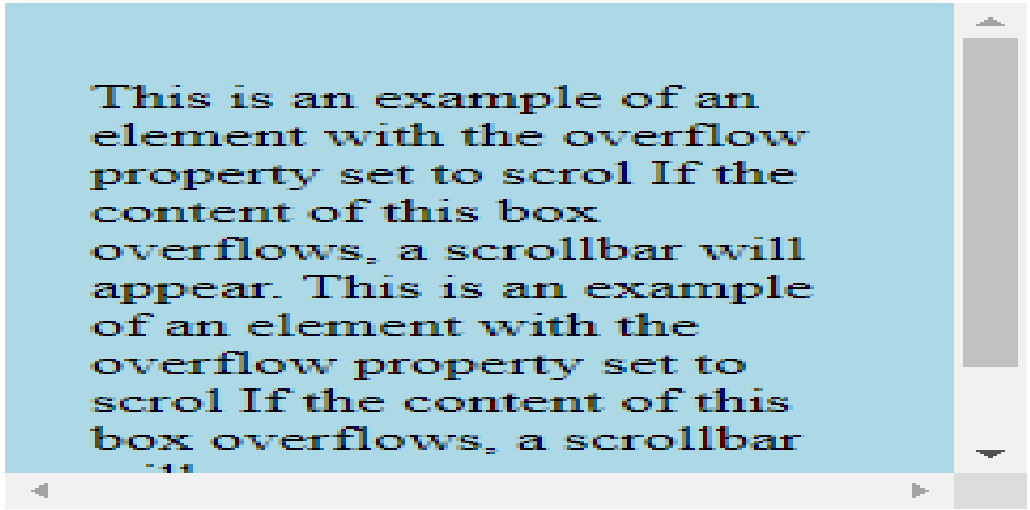

This is an example of an element with the overflow property set to scrol If the content of this box overflows, a scrollbar will appear. This is an example of an element with the overflow property set to scrol If the content of this box overflows, a scrollbar will appear
dffdfdffdfdfdf

Overflow

- In CSS, the **overflow** property is used to **control what happens when the content of an element overflows** its allocated space within the box model.

Here are some possible values for the overflow property:

- **scroll:** Content that overflows is clipped, and a **scrollbar is added** to allow scrolling to see the rest of the content.




```
<!DOCTYPE html>
<html>
<head>
        <title>Overflow Example</title>
        <style>
                .box {
                background-color: lightblue;
                padding: 20px;
                width: 200px;
                height: 200px;

                overflow: scroll;
                }
        </style>
</head>
<body>  <div class="box">
                <p>This is an example of an element with the overflow property set to scroll. If the content of this box overflows, a scrollbar will appear.</p>
        </div>
</body>
</html>
```

Resetting Defaults

- Resetting defaults in CSS can be achieved by using a **CSS reset stylesheet**.
- A CSS reset stylesheet is a set of CSS rules that are designed to **reset or override the default styles** applied by browsers to HTML elements.
- Here's an example of a simple CSS reset stylesheet:

```
/* CSS Reset */
html, body, div, span, applet, object, iframe,
h1, h2, h3, h4, h5, h6, p, blockquote, pre,
a, abbr, acronym, address, big, cite, code,
del, dfn, em, img, ins, kbd, q, s, samp,
small, strike, strong, sub, sup, tt, var,
b, u, i, center,
dl, dt, dd, ol, ul, li,
fieldset, form, label, legend,
table, caption, tbody, tfoot, thead, tr, th, td {
    margin: 0;
    padding: 0;
    border: 0;
    font-size: 100%;
    font: inherit;
    vertical-align: baseline;
}
```

```
/* HTML5 display-role reset for older browsers */
article, aside, details, figcaption, figure,
footer, header, hgroup, menu, nav, section {
    display: block;
}
body {   /* Body styles */
    line-height: 1;
}
a {/* Links */
    color: #000;
    text-decoration: none;
}
a:hover {
    text-decoration: underline;
}
```

Visibility

- The **visibility** property in CSS is used to **control the visibility of an element**.

The visibility property can have the following values:

- **visible:** The element is visible.
- **hidden**: The element is hidden, but still takes up space.
- **collapse:** Only for table elements. The **row or column is removed from the table layout** and the **space is made available for other content.**

```
<!DOCTYPE html>
<html>
<head>
        <title>Visibility Example</title>
        <style>
            .box {
                background-color: lightblue;
                padding: 20px;
                width: 200px;
                height: 200px;
                visibility: hidden;
                }
        </style>
</head>
<body>
        <div class="box">
                <p>This is an example of an element with the visibility property set to hidden. The element is hidden, but still takes up space.</p>
        </div>
</body>
</html>
```

Visibility

- In this example we set the visibility property to hidden.
- When you view this code in a web browser, you will **see a box with light blue background color and 20 pixels of padding,** but there will be **no content visible inside the box**.
- This is because the visibility property has been set to hidden, which hides the content of the element but still takes up space.

```
<!DOCTYPE html>
<html>
<head>
        <title>Visibility Example</title>
        <style>
            .box {
                background-color: lightblue;
                padding: 20px;
                width: 200px;
                height: 200px;
                visibility: hidden;
                }
        </style>
</head>
<body>
        <div class="box">
                <p>This is an example of an element with the visibility property set to hidden. The element is hidden, but still takes up space.</p>
        </div>
</body>
</html>
```

CSS The display Property

- The display property is an important CSS property for controlling layout. It specifies whether an HTML element is treated as a block or an inline element.

- Every HTML element has a default display value, depending on what type of element it is. The default display value for most elements is block or inline.

- The display property is used to change the default display behavior of HTML elements.

Block-level Elements

A block-level element ALWAYS starts on a new line and takes up the full width available (stretches out to the left and right as far as it can).

Examples of block-level elements:

-
- <h1> - <h6>
- <p>
- <form>
- <header>
- <footer>
- <section>

Inline Elements

An inline element DOES NOT start on a new line and only takes up as much width as necessary.

This is an inline element inside a paragraph.

Examples of inline elements:

-
- <a>
- <img>

Common display Values

The CSS display property has many values. The following table lists the most commonly used:

Value	Description
inline	Displays an element as an inline element
block	Displays an element as a block element
contents	Makes the container disappear, making its child elements children of the element the next level up in the DOM
flex	Displays an element as a block-level flex container
grid	Displays an element as a block-level grid container
inline-block	Displays an element as an inline-level block container. The element itself is formatted as an inline element, but you can apply height, width, padding, and margin values
none	The element is completely hidden from the document flow (does not take up any space).

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  background-color: blue;
  padding: 10px;
}

span {
  background-color: red;
  color: white;
  padding: 10px;
}

p.ex1 {display: none;}
p.ex2 {display: inline;}
p.ex3 {display: block;}
p.ex4 {display: inline-block;}
p.ex5 {display: flex;}
p.ex6 {display: grid;}
</style>
</head>
<body>
<h1>The display Property</h1>

<h2>display: none:</h2>
<p class="ex1">
  <span>Text 1</span>
  <span>Text 2</span>
  <span>Text 3</span>
</p>

<h2>display: inline:</h2>
<p class="ex2">
  <span>Text 1</span>
  <span>Text 2</span>
  <span>Text 3</span>
</p>

<h2>display: block:</h2>
<p class="ex3">
  <span>Text 1</span>
  <span>Text 2</span>
  <span>Text 3</span>
</p>

<h2>display: inline-block:</h2>
<p class="ex4">
  <span>Text 1</span>
  <span>Text 2</span>
  <span>Text 3</span>
</p>

<h2>display: flex:</h2>
<p class="ex5">
  <span>Text 1</span>
  <span>Text 2</span>
  <span>Text 3</span>
</p>

<h2>display: grid:</h2>
<p class="ex6">
  <span>Text 1</span>
  <span>Text 2</span>
  <span>Text 3</span>
</p>

</body>
</html>
```

The display Property

display: none:

display: inline:

Text 1 Text 2 Text 3

display: block:

Text 1 Text 2 Text 3

display: inline-block:

Text 1 Text 2 Text 3

display: flex:

Text 1 Text 2 Text 3

display: grid:

Text 1

CSS The position Property

The `position` property specifies the positioning type for an element.
This property can have one of the following values:

- `static` - This is default. Element is positioned according to the normal document flow
- `relative` - Element is positioned relative to its normal position in the document flow
- `fixed` - Element is positioned relative to the viewport
- `absolute` - Element is positioned relative to the nearest positioned ancestor
- `sticky` - Element toggles between a relative and fixed position, depending on the scroll position

Elements are then positioned to their final location with the `top`, `bottom`, `left`, and `right` properties.

Examples

Static

```
<!DOCTYPE html>
<html>
<head>
<style>
div.static {
  position: static;
  border: 3px solid #73AD21;
}
</style>
</head>
<body>

<h2>Using position: static;</h2>

<p>An element with position: static; is always positioned according to the normal flow of the page:</p>

<div class="static">
This div element has position: static;
</div>

</body>
</html>
```

Relative

```
div.relative {
  position: relative;
  left: 30px;
  border: 3px solid #73AD21;
}
```

Fixed

```
div.fixed {
  position: fixed;
  bottom: 0;
  right: 0;
  width: 300px;
  border: 3px solid #73AD21;
}
```

```
div.relative {
  position: relative;
  width: 400px;
  height: 200px;
  border: 3px solid green;
}

div.absolute {
  position: absolute;
  top: 80px;
  right: 0;
  width: 200px;
  height: 100px;
  border: 3px solid red;
}
```

absolute

CSS position: sticky

- An element with `position: sticky;` toggles between a `relative` and `fixed` position, depending on the scroll position.
- A sticky element is positioned relative until a certain scroll position is reached - then it "sticks" in that place (like position:fixed).

Sticky vs. Fixed
The key differences between `sticky` and `fixed` positioning:

Feature	position: sticky	position: fixed
Initial behavior	Acts like relative	Always fixed to viewport
Space in document	Takes up space initially	Does not take up space
Scroll behavior	Sticks after threshold	Always fixed
Container bound	Yes, respects parent boundaries	No, always relative to viewport

```
<!DOCTYPE html>
<html>
<head>
<style>
div.sticky {
  position: sticky;
  top: 0;
  padding: 5px;
  background-color: #cae8ca;
  border: 2px solid #4CAF50;
}
</style>
</head>
<body>

<h2>Using position: sticky;</h2>

<p>Try to scroll this page to understand how sticky works.</p>

<p>Here, when the sticky element reach the top of the page (top: 0), it sticks to this position!</p>

<div class="sticky">I have position sticky!</div>

<div style="padding-bottom:2000px">

<p>Scroll back up to remove the stickyness!</p>
<p>Some text to enable scrolling.. Lorem ipsum dolor.</p>
<p>Some text to enable scrolling.. Lorem ipsum dolor efficiantur his ad. Eum no molestiae voluptatibus.</p>
<p>Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset.</p>
</div>

</body>
</html>
```

CSS Practical Questions

Practical 1: Introduction to CSS
Aim: Apply CSS using Inline, Internal, and External stylesheets.
Task:
- Create a webpage with a heading, paragraph, and image.
- Apply:
    - Inline CSS to the heading
    - Internal CSS to the paragraph
    - External CSS to the image

Practical 2: CSS Selectors
Aim: Implement different CSS selectors.
Task:
Create a webpage containing headings, paragraphs, buttons, and links.
Apply:
- Universal Selector
- Element Selector
- Class Selector
- ID Selector
- Group Selector
- Attribute Selector

CSS Practical Questions

Practical 3: Colors and Backgrounds
Aim: Apply colors and background properties.
Task:
- Create five boxes.
- Apply:
    - Different background colors
    - Background image
    - Gradient background
    - Background repeat
    - Background size
    - Background position

Practical 4: Typography
Aim: Apply text and font properties.
Task:
Design a webpage using:
- Different font families
- Font sizes
- Font weights
- Text alignment
- Letter spacing
- Word spacing
- Text transformation
- Text shadow

Practical 5: Borders and Border Radius
Aim: Design attractive cards.
Task:
Create four cards using:
- Solid border
- Dashed border
- Dotted border
- Rounded corners
- Box shadow

Practical 6: Margin and Padding
Aim: Understand spacing.
Task:
Create nested boxes showing:
- Margin
- Padding
- Border
- Content area

Display the effect of changing each property.

Practical 7: CSS Box Model
Aim: Demonstrate the CSS Box Model.
Task:
Create a webpage illustrating:
Content
Padding
Border
Margin
Calculate the total width and height of the element.

Practical 8: Display Properties
Aim: Study display properties.
Task:
Create examples using:
block
inline
inline-block
none
flex
grid
Compare their behavior.

Practical 9: Mini Project
Title: Responsive College Website
Requirements:
- Responsive Navigation Bar
- Hero Section
- About College
- Courses
- Faculty
- Gallery
- Admission Form
- Contact Us
- Footer
- CSS Animations
- Media Queries
- Hover Effects
- Flexbox/Grid Layout